

2023年

情報科学実験 I 指導書



A コンテンツ制作と音声処理



B 画像処理と生成



C 通信ネットワーク



D データ科学と自動制御



愛知県立大学 情報科学部

Aichi Prefectural University

自動制御—モータ制御を通したコントローラ設計体験

1 はじめに

1.1 本実験の目的

本実験の目的は、自動制御の例として、モータ制御を通してコントローラ設計を体験することである。簡易なモータ実験装置を Arduino を介して MATLAB/Simulink から制御する。別の側面は、理論と実際とのつながり、最近のラビッドプロトタイピングを垣間見ることである。ここで得た知識や経験が卒業研究等に生きることを期待する。

1.2 予習（事後レポート）について

実験は、第 1 週に 2～3 節、第 2 週に 4.1～4.2 節、第 3 週に 4.3 節と取り組むのが目安である。実験を有意義かつ効率的に進めるため、あらかじめ各範囲をよく読み、実験内容を把握されたい。本文中の「予習課題」に対しては、事前レポートを手書きで作成し、各週開始時間に提出せよ。第 1 週用は「予習 1」, 「予習 2」, 第 2 週用は「予習 4」, 「予習 5」である。「自由」のついた「予習 3」は任意課題である。

1.3 持ち物と心構え

実験には、実験指導書、筆記用具、ノート（実験中や調査時のメモ、課題や考察の検討のために）を持参して臨むこと。実験中に得たデータは、演習室アカウントに保存するだけでなく、クラウドストレージや USB メモリなどにもバックアップできるよう準備しておくこと。

本実験は電気・機械要素を扱う実験である。安全に実施するため以下 5 点には特に留意すること：

- 1) 机上进行して実験スペースを確保する。特に、USB ケーブルの引き回しに注意する。
- 2) 配線変更、離席の場合は、必ず電池ボックスを OFF, USB ケーブルを PC から抜いた状態にする。
- 3) 濡れた手はもちろん、乾いた手であっても電源の入った回路の端子に触れない。
- 4) 回路内の注目対象（今回の場合はモータ）に対し、電源の正（プラス）側は暖色ケーブルで、負（マイナス）側は寒色ケーブルでそれぞれ配線して、視覚的な確認を容易にする。
- 5) モータ軸など可動部を手で触れたり、何かが巻き込まれないようにする。

なお、実験装置が故障した可能性がある場合には必ず申し出ること。

1.4 レポート作成について

レポートでは第三者に対し、各実験の目的と方法、得られた結果、理論と結果を対比した考察といったものを正確に伝える必要がある。レポート作成にあたって指導書・図書・論文などから得た知識は、それを理解したうえで自分の表現で記述し、必ず出典（書誌情報）を明らかにして参照・引用せよ。実験内容や課題に関する議論・相談は積極的に行っているが、過去のものを含め、他人のレポートを共有する（見る／見せる）、行為は、試験のカンニングに相当するので絶対に避けること。

本実験テーマでは、事前レポート（§ 1.2 参照）と事後レポートの 2 種類を課す。事後レポートの範囲、提出期限、媒体、場所は以下の通りである：

レポート範囲	期限	媒体	場所
第 1 週分「課題 1」～「課題 5」	第 3 週の実験開始時間	紙	実験する演習室の教卓
第 2, 第 3 週分「課題 6」～「課題 10」	第 3 週の日曜日まで	PDF	UNIPA 課題

レポートの形式や書式に不備がある場合、提出後の実験中に、該当レポートを返却して修正&再提出を求めることがある。レポート作成上の注意点を以下に挙げる（形式や書式を見直すためのチェックリストとして活用されたい）：

- 全般や形式について
 - レポート作成のポイントは抑えている。そうでなければ、たとえば文献 [1, 2, 3, 4] あたりを読むこと。
 - $\text{\LaTeX}$ で作成すること。各ページにはページ番号を振り、両面印刷すること。
 - A4 用紙は長辺を縦になるように使用すること。
 - 独立した表紙は必要ない。冒頭にタイトル、学籍番号・氏名を明記したあと、本文を続ければよい。
 - レポートの基本構成は、「実験目的」「(課題に対する) 考察」「結論」「参考文献リスト」とせよ。「考察」においては、必要に応じて関連する「理論」や「実験の方法と結果」をまじえて論理的に説明せよ。
- 図、表、参考文献について
 - 図には図の下部に固有の番号とタイトル（例：「図 1 〇〇に関する実験結果」）をつけること。
 - 表には表の上部に固有の番号とタイトル（例：「表 3 △△で使用したパラメータ」）をつけること。
 - レポートに掲載するグラフは視覚的な区別が容易となるよう配色や線種を検討せよ。カラー作成する場合は実線を用い、モノクロ作成する場合は、実線、破線、一点鎖線など線種を使い分けること。
 - グラフで複数条件を比較する場合、重ね描きを基本とせよ。重ね描きによる区別が困難な場合は、視覚的に公平な比較ができるよう各軸の表示範囲に注意して並べること。
 - グラフでは、各軸にそれが表す数量の名称と単位を明記すること。
 - 表では見出し欄にその列／行が表す名称のほか、単位があるものはそれを明記すること。
 - 本文中では、例えば「表 3 に示すパラメータで実験した結果を図 4 に示す」のように、図・表の番号をもとに引用すること。「左図のように」「つぎのページに示す表のように」といった表現は不適切である。
 - 参考文献リストの各文献には固有の番号を振ること。また、各文献の出典（書誌情報）は、「著者・編者名：「タイトル」、シリーズ名、巻・号、該当ページ、発行所、発行年。」を参考書式として整えること。（例：[2] 高木 貞治：“解析概論”，改訂第三版（軽装版），pp.386–389，岩波書店，1983.）
 - 本文中では「ストークスの定理 [2] より次式が成り立つ」のように文献番号をもとに引用すること。
 - 指導書の図、表を引用する場合は、参考文献リストに指導書を加えたうえで、「文献 [3] の図 5 のように配線した」のように記述すること。

2 Arduino と MATLAB/Simulink の準備

2.1 Arduino とは

我々が日頃の生活で利用する家電製品や生産現場で用いられる精密機械・産業用機器を支えるモノの一つとして、マイクロコンピュータ（通常マイコンと呼ばれる）がある。マイコンとは超小型でありながら、中央処理装置（CPU）、記憶装置や入出力装置などコンピュータの諸機能をそなえたコンピュータである。

アルドゥイーノ
Arduino はオープンソースのプロトタイピング環境であり、マイコンの Arduino ボードとソフトウェア開発環境の **Arduino IDE** からなる [5]。Arduino の動作にはスケッチと呼ばれるプログラムが必要となるが、電子工学やプログラミングに不慣れな初心者でも開発環境の Arduino IDE を用いて比較的容易に作ることができるよう工夫されている。

イタリアで開発された Arduino は、入出力ポートの使いやすさと安価であることから瞬く間に世界中に広く普及し、Internet of Things (IoT)、ウェアラブル機器や組み込み環境などの発展に寄与している。Arduino の普及には、ニューヨーク大学大学院でインタラクションデザインの教授法の一つとして考案されたフィジカルコンピューティング (**Physical Computing**) が関係する。フィジカルコンピューティングでは、電子工作やマイコンのプログラミングを通じて原理原則を身体で覚えることからはじめ「人々がいかにコンピュータとコミュニケーションし得るか？」について考え直すことを提案している [6]。

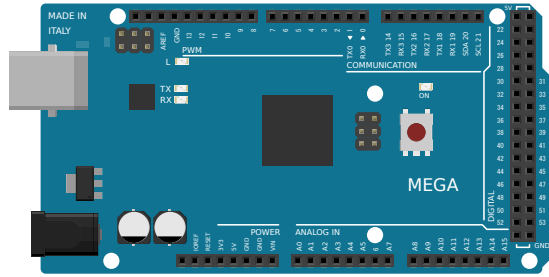


図 1: Arduino Mega 2560



(a) 起動後のウィンドウ

(b) ボード選択時

(c) 書き込みが正常終了した場合

図 2: Arduino IDE

Arduino ボードには様々なタイプがあるが、本実験では図 1 に示す Arduino Mega 2560 を用いる。

2.2 Arduino IDE の設定

Arduino IDE は Arduino の開発環境である。ドライバソフトウェアとともに Arduino のソフトウェアに含まれ、Arduino のホームページ (<http://www.arduino.cc>) で提供されている。同ページメニューバーの“Software”から Windows 版, macOS 版, Linux 版が入手でき、2023 年 9 月 25 日現在 Ver. 2.2.1 が最新である。

ターミナルでつぎのコマンドを実行して、Arduino IDE を起動する：

```
$ arduino
```

起動後のウィンドウの様子を図 2a に示す。メニューバーの左下に 5 つのボタンが並んでいる。左から順に「Verify」「Upload」「New」「Open」「Save」であり、各機能は表 1 のとおりである。

Arduino IDE を利用するにあたり、Arduino ボードの種類、プロセッサの種類、シリアルポート番号を設定する必要がある。本実験では Arduino Mega 2560 を使用するため、Arduino IDE を以下のように設定する：

1. メニュー [ツール] - [ボード] から [Arduino Mega or Mega 2560] を選ぶ (図 2b)。
2. メニュー [ツール] - [プロセッサ] から [ATmega2560 (Mega 2560)] を選ぶ。
3. メニュー [ツール] - [シリアルポート] から [/dev/ttyACM0] を選ぶ。

表 1: Arduino IDE のボタンとその意味

ラベル	マーク	意味
Verify	チェック	スケッチの文法チェックとコンパイルを実行する。
Upload	右矢印	スケッチを Arduino へ書き込む。
New	ノート	新しいスケッチを開く。
Open	上矢印	スケッチを開く。
Save	下矢印	スケッチを保存する。

2.3 ArduinoIO の設定

ArduinoIO は、MATLAB/Simulink から Arduino の入出力を直接扱えるよう開発されたライブラリである。このライブラリを用いることで Arduino 用のプログラミングや Arduino への書き込みは不要となり、Arduino を単なる IO ボードとして PC 上で MATLAB/Simulink による計算・処理が可能となる。厳密性を期待しなければ、Simulink の Real-Time Pacer ブロックを利用することによってある程度のリアルタイム性も確保できる。

MATLAB/Simulink と Arduino 入出力ポートとの通信のため、以降の実験を前に 1 度だけ、ArduinoIO に含まれる専用スケッチを Arduino へ転送しておく必要がある。手順は以下のとおりである：

1. Arduino と PC とを USB ケーブルで接続する。
2. Arduino IDE を起動する。
3. [ファイル] – [開く] からつぎのスケッチを開く：

```
/cis/public/sa/local/MATLAB/arduino-1.6.0/ArduinoIO/pde/adioes/adioes.pde
```

4. Arduino IDE ウィンドウ左上の「検証」ボタン（チェックマーク）をクリックして、正常にコンパイルができることを確認する。
5. 検証ボタンの右に配置されている「マイコンボードに書き込む」ボタン（右矢印）をクリックし、スケッチを Arduino へ転送する。書き込みが正常に終了すると、ウィンドウ下部に「マイコンボードへの書き込みが完了しました。」というメッセージとともに転送したスケッチのデータサイズが表示される（図 ??）。

Arduino IDE が利用できる演習室環境では、ArduinoIO ライブラリは MATLAB R2016a, R2018b, および R2021b で使用できる状態にあるⁱ⁾。matlab2016a コマンド、matlab2018b コマンド、あるいは matlab コマンドで MATLAB を起動し、コマンドラインで

```
» simulink ↵
```

とすると出現する [Simulink ライブラリ ブラウザ] ウィンドウから [Arduino IO Library] を選択すると図 ??に示すブロック群を確認できる。MATLAB コマンドラインで

```
» arduino_io_lib ↵
```

とすると、Arduino IO Library のみのウィンドウが現れる。

ⁱ⁾ これら以外のバージョンでは AudinoIO ライブラリが利用可能でないので注意されたい。

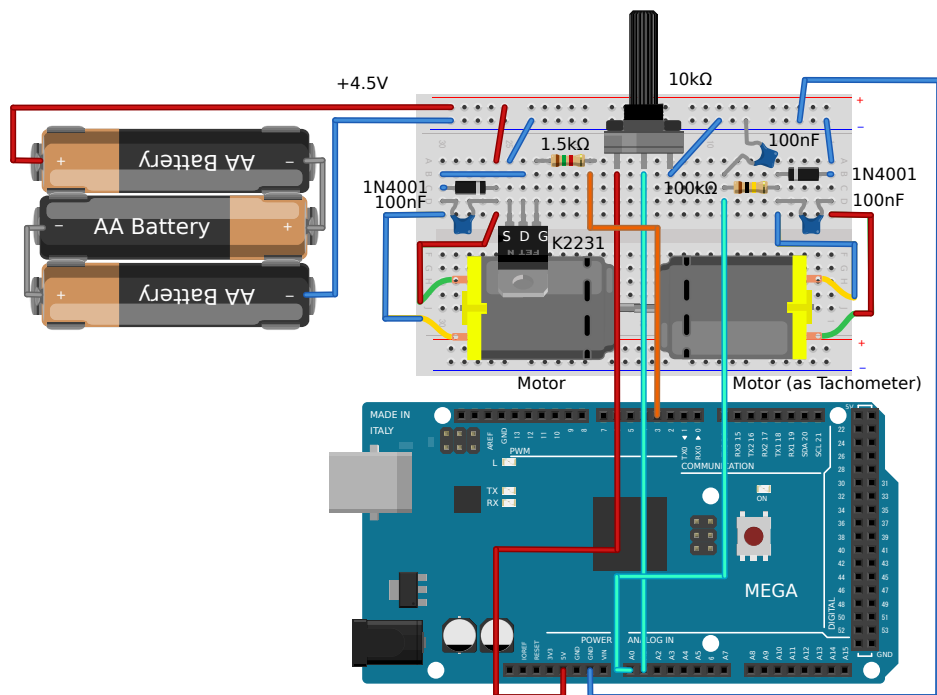


図 3: モータの速度制御系（実体配線図）

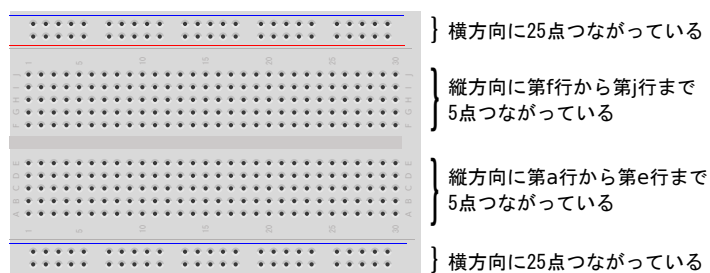


図 4: ブレッドボード

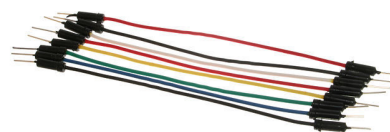


図 5: ジャンパワイヤⁱ⁾

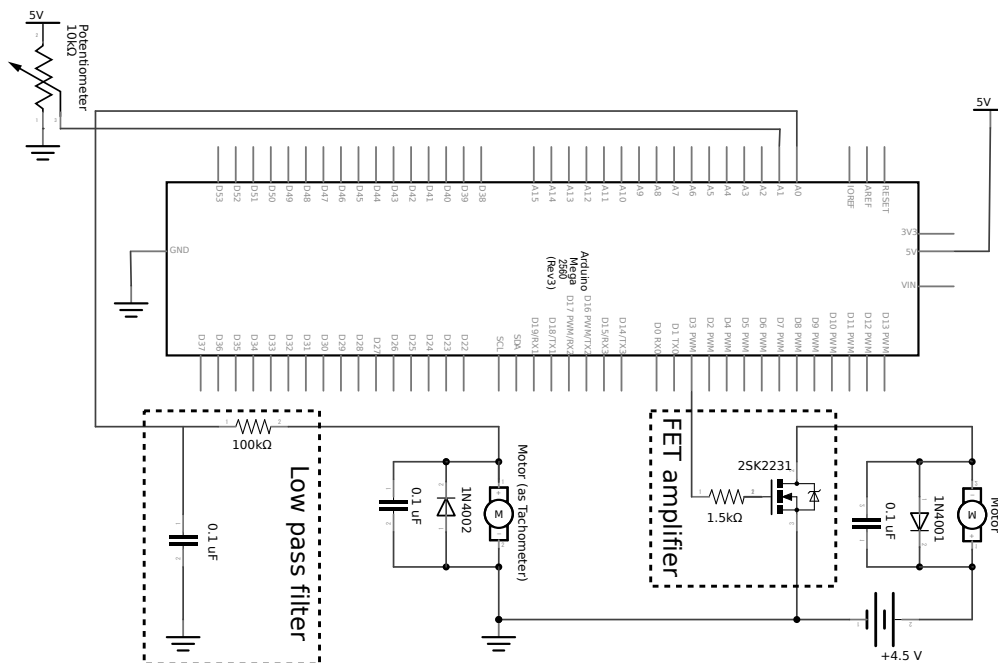
3 モータの速度制御実験（モデルを用いない場合）

3.1 実験装置の準備

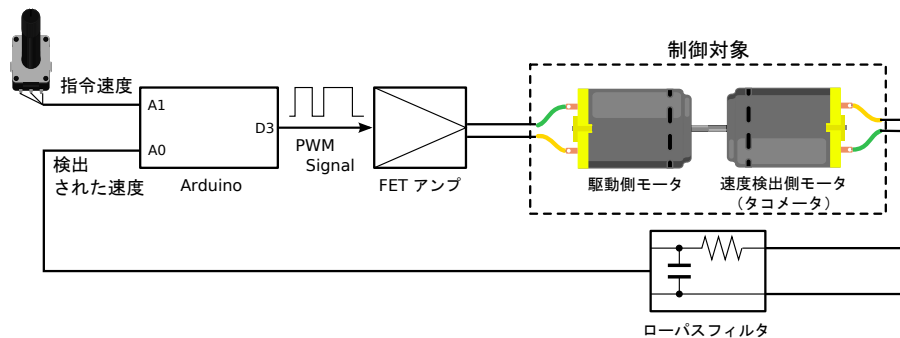
実験装置を図 3 のように構成する。時間の都合から、モータや各種素子はブレッドボード（図 4）に配置済みである。ブレッドボード上の配置に誤りがないか、電気的な接続に注意しながら確認する。問題がなければ、Arduino とブレッドボードとの接続をジャンパワイヤ（図 5）を用いて行い、電池を入れた電源をブレッドボードに接続する。このとき、電源スイッチが OFF であることを確認すること。電源スイッチを ON にするのは各実験を実行する直前のみとし、実行完了次第 OFF にする。

図 6 に、構成した実験装置に対応する回路図と簡易ブロック線図を示す。Arduino のアナログ出力 D3 から出力された Pulse Width Modulation (PWM) 波形で FET（電界効果トランジスタ）を ON/OFF スイッチングし、その電圧でモータを駆動する。Arduino では、一般的なアナログ出力のように出力を 0V から 5V まで連続的に変化させることはできないため、PWM が採用されている。回路を単純化するため、モータの回転方向は一方方向に制限している。タコメータの出力はノイズの影響を低減化するため RC ローパスフィルタを経由し、アナログ入力 A0 へ接続する。また、速度指令を外部から与えられるよう、可変抵抗器（ポテンシオメータ）の出力をアナログ入力 A1 に接続する。

ⁱ⁾ https://commons.wikimedia.org/wiki/File:A_few_Jumper_Wires.jpg より引用（最終アクセス日：2022 年 9 月 20 日）。



(a) 回路図



(b) 簡易ブロック線図

図 6: モータの速度制御系

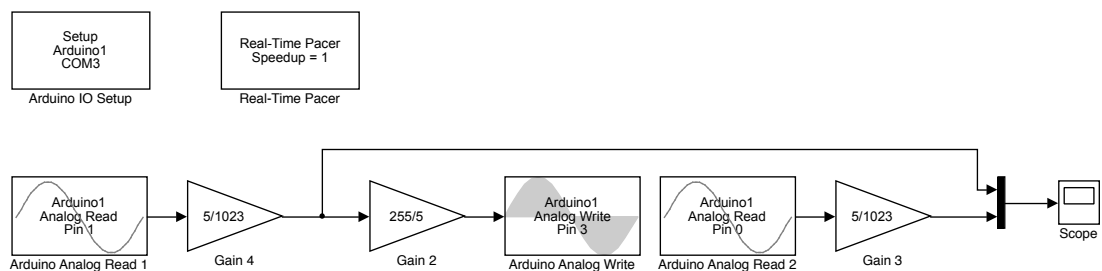


図 7: 動作テスト用 Simulink モデル

- 予習 1 本実験で対象とする DC モータは、どのような仕組み・原理で動作するのか、図書やウェブなどで調査し、数式と図の両方を用いて説明せよ。
- 予習 2 モータの駆動に用いる PWM の原理について、図書やウェブなどで調査し、数式と図の両方を用いて説明せよ（キーワード：デューティ比）。また、PWM にすることの意義（メリット）を合わせて述べよ。

表 2: モータの動作テストにおけるパラメータ設定

設定項目	選択肢／設定値	備考
《コンフィギュレーション パラメータ ⁱⁱⁱ⁾ 》		
(シミュレーション時間)		
開始時間	0.0	
終了時間	10.0	適宜変更
(ソルバーオプション)		
タイプ	固定ステップ	
ソルバー	離散 (連続状態なし)	
《Arduino IO Setup ブロック》		
Serial (COM) port	/dev/ttyS101	各環境のシリアルポート番号を指定
《Real-time Pacer ブロック》		
Speedup (Simulation Time/Real Time)	1	
《Arduino Analog Read 1 ブロック》		
Pin	1	ポテンシオメータの電圧を読み取る
Sample Time	ts	
《Arduino Analog Read 2 ブロック》		
Pin	0	タコメータの電圧を読み取る
Sample Time	ts	
《Arduino Analog Write 1 ブロック》		
Pin	3	駆動用モータへの PWM 出力
Sample Time	ts	
《Gain 2 ブロック》		
ゲイン	255/5	
《Gain 3 ブロック》		
ゲイン	5/1023	
《Gain 4 ブロック》		
ゲイン	5/1023	

3.2 モータの動作テスト

動作の確認と慣らし ⁱⁱ⁾のため、モータを駆動させてタコメータの電圧がどのように変化するか観察する。テスト用 Simulink モデル `velo_test.mdl` を開く (図 7)。このモデルでは、ポテンシオメータの電圧をアナログ入力 A1 から読み取り、その値を 0～5 V の電圧に変換し、デジタル出力 D3 から駆動用モータへ出力する。主なパラメータを表 2 に示す。特に、サンプリング周期 `ts` はつぎの MATLAB コマンドによりあらかじめ値を設定しておく：

```
» ts = 1/50
```

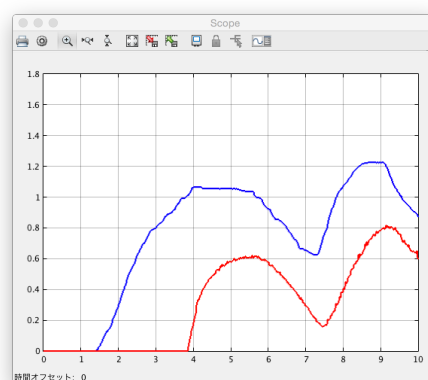


図 8: 動作テストの実験結果例

ⁱⁱ⁾ 各週の実験開始時にこの動作テストを 2～3 回実行すると、モータの動作が良好になる傾向にある。

ⁱⁱⁱ⁾ メニューから [シミュレーション] - [コンフィギュレーション パラメータ] を選択する。

なお、PC の性能・負荷状況によっては 50 Hz のサンプリング周波数は速すぎることがある。Simulink モデルを実行した際に、実際の時間よりも長くかかる場合は、サンプリング周波数を（たとえば 40 Hz にするなど）少し下げて様子を見ること。ゲインブロックの値は、Arduino の電源電圧やモータ駆動用電源の電圧を 5 V と仮定して、アナログ入出力値と実際の電圧を一致させるためのものである。

ポテンシオメータを時計回りに回しきり、電源を ON にしたら、メニューから [シミュレーション] - [実行] を選択、あるいは実行ボタン  をクリックして、Simulink モデルを実行する。ウィンドウ左下にシミュレーション状態が表示されるが、「コンパイル」から「初期化中」に移行してしばらく（15~20 秒間）待つと「実行中」となる。ポテンシオメータをゆっくり回して速度が変化することを確認する。Scope に表示される出力波形の一例を図 8 に示す。シミュレーション終了時間になると「終了中」「準備完了」と表示され、つぎの実行が可能な状態となる。

3.3 比例制御（P 制御）

フィードバック制御は、制御対象の出力をセンサなどで観測し、それと目標値（指令値）との差（偏差）に応じて制御入力を決めるという動作を連続的に行うことで実現できる（図 9）。偏差から制御入力を決める部分は制御器（あるいは補償器）と呼ばれ、これをいかに設計するかが制御系設計において特に重要である。

予習 3 [自由] 古典的、かつ代表的なフィードバック制御の方法として、比例-積分-微分制御（いわゆる PID 制御）がある。PID 制御の基本的な考え方、メリット、デメリットをわかりやすく説明せよ。

もっともシンプルな制御方法は、比例（**Proportional**）制御（通例 **P** 制御と略す）である。P 制御器を実装した Simulink モデル `velo_p.mdl` を開く（図 10）。駆動モータとタコメータからなる制御対象の部分は表現を簡単化するためサブシステム化（ブラックボックス化）していることに注意されたい。Motor ブロックをダブルクリックすれば、内部構造にアクセスでき、図 7 の一部と同じであることがわかる。

目標値は、(1) ポテンシオメータのつまみをわまして外部から手で与えるモードと、(2) パルス状に与えるモードとを、Manual Switch ブロックで切り替えられるようになっている。主なパラメータを表 3 に示す。

ところで、本実験のフィードバック制御系は、タコメータに DC モータを使用しているため、実際の速度とタコ

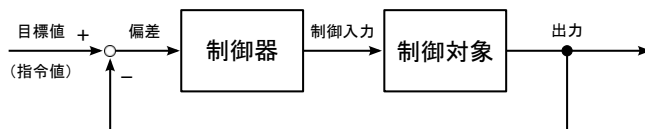


図 9: フィードバック制御系の基本形式

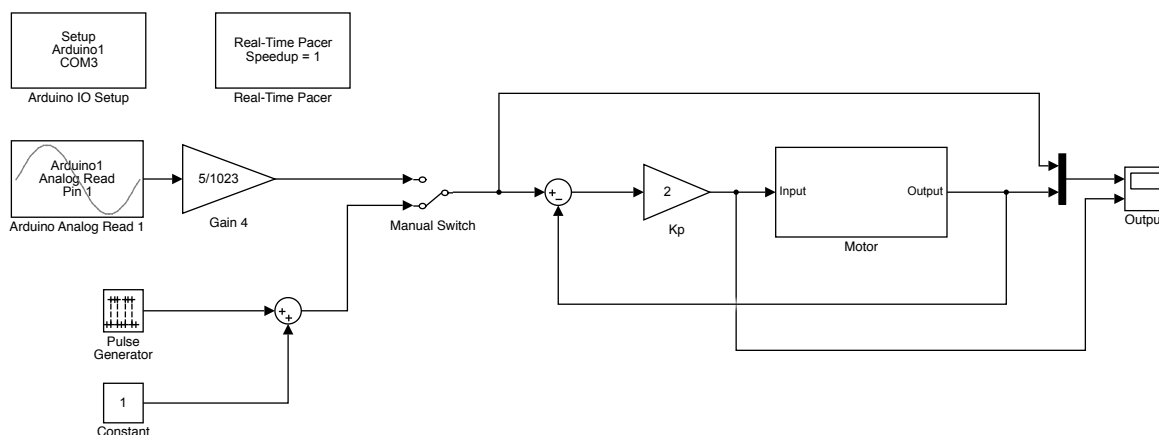


図 10: 比例制御系の Simulink モデル（ブロック線図全体）

表 3: 比例制御系におけるパラメータ設定

設定項目	選択肢／設定値	備考
《Pulse Generator ブロック》		
パルスタイプ	サンプルベース	
振幅	0.5	
周期 (サンプル数)	4/ts	整数値になるよう設定
パルス幅 (サンプル数)	2/ts	同上
サンプル時間	ts	ts の値は MATLAB コマンドラインで設定
《Constant ブロック》		
定数値	1	

メータの発生電圧との関係がわからない^{iv)}。回転計があれば回転速度と電圧との関係を調べられるが、ここでは回転速度の代わりにタコメータの出力電圧をそのままフィードバックすることにし、目標値も電圧とする。したがって、タコメータの出力電圧を目標電圧に追従させる制御系を考えることとなる。

比例ゲイン K_P は、なるべく小さな値から試しはじめて、目標値に対する出力の追従特性を見ながら調整されたい。比例ゲインを大きくしすぎると、制御入力に 0~5 V の間におさまらず飽和してしまう。できる限り、飽和しない範囲で比例ゲインを調整すること。図 11 は Scope に表示される実験結果の一例 ($K_P = 4$ の場合) である。以降の実験では、各課題に対する考察が必要となりそうなデータをファイルとして保存し、考察内容にあわせて適切なグラフを作成すること (具体的な方法は付録 A 参照)。予備的に Scope 表示をスクリーンショット保存してもよいが、それらをレポートで使用することは禁じる。

課題 1 比例ゲイン K_P をさまざまに変えると、目標値と出力との差 (偏差) はどのように変化するか。複数の実験結果から洞察される、 K_P と偏差との関係性を説明せよ (ヒント: 最もよい結果の探索を要求しているわけではない。ある値基準に K_P を小さくしたり、大きくしたりして、3 通り以上の特徴的な結果を比較せよ)。

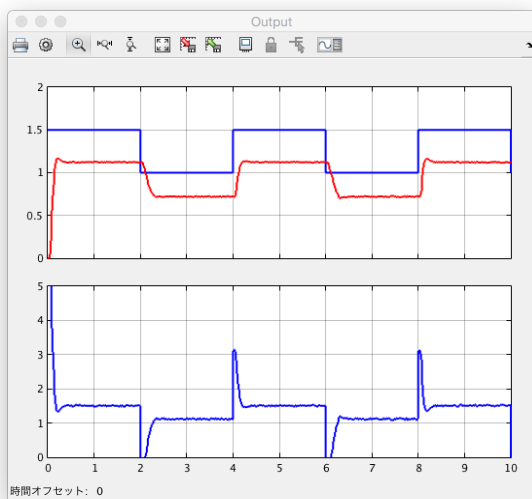


図 11: P 制御による実験結果^{v)}

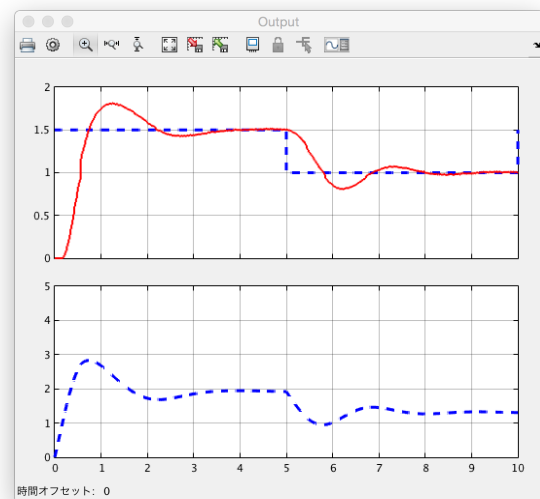


図 12: I 制御による実験結果^{v)}

^{iv)} 市販のタコメータは、電圧と回転速度は高精度に比例し、両者間の比例定数も仕様書に明記されている。

^{v)} 各図の上段は目標値 (青色) と出力 (赤色) の時間変化を、下段は制御入力の時間変化を、それぞれ表す。

表 4: 積分制御系におけるパラメータ設定

設定項目	選択肢／設定値	備考
《Pulse Generator ブロック》		
周期 (サンプル数)	10/ts	積分制御では応答が遅いので
パルス幅 (サンプル数)	5/ts	同上

3.4 積分制御 (I 制御)

比例制御では時間が経過しても、目標値と出力との差が小さくならず、一定の偏差が残る（この偏差を定常偏差という）。比例制御によって定常偏差をゼロにすることはできない。その理由はつぎのように説明できる。目標値と出力との差を e 、制御入力を u 、比例ゲインを K_P とすると、比例制御系では次式の関係にある：

$$u(t) = K_P e(t). \quad (1)$$

定常偏差がゼロ、つまり $e = 0$ のとき、(1) 式から制御入力がゼロであることを意味する。一方、モータを一定回転数で回し続けるには、制御入力（電圧）は必要、すなわち $u \neq 0$ でなければならない。これは明らかな矛盾であり、比例制御で定常偏差をゼロにできないことを説明している。

これに対して積分（Integral）制御（通例 I 制御と略す）が有効である。積分制御では、偏差の積分に積分ゲイン K_I をかけたものが制御入力となる。つまり、次式の関係になる：

$$u(t) = K_I \int_0^t e(\tau) d\tau. \quad (2)$$

制御入力が偏差の時間積分に基づいて決まるため、ある時刻で偏差がゼロでも、そのときの制御入力はゼロにはならない。この性質から、偏差をゼロにしつつ、制御入力を非ゼロにすることが可能となる。

偏差が生じている限り、それが時間積分されて制御入力はどんどん大きくなる。その増大は偏差がゼロになってはじめて止まる。制御系が安定になる（発散せず、ある値に収束する）ように設計しておけば、時間の経過とともに定常偏差はゼロに収束する。これが、積分制御によって定常偏差がゼロになる仕組みである。

比例制御の Simulink モデル `velo_p.mdl` に、積分器と積分ゲイン K_I を追加した図 13 の Simulink モデル `velo_pi.mdl` を開く。主なパラメータは表 4 のとおり設定する。図 12 は実験結果の一例である。

課題 2 比例ゲインを $K_P = 0$ として、積分ゲイン K_I をさまざまに変えたとき、 K_I と偏差との関係性を説明せよ。なお、比例制御との違いを強調すること。

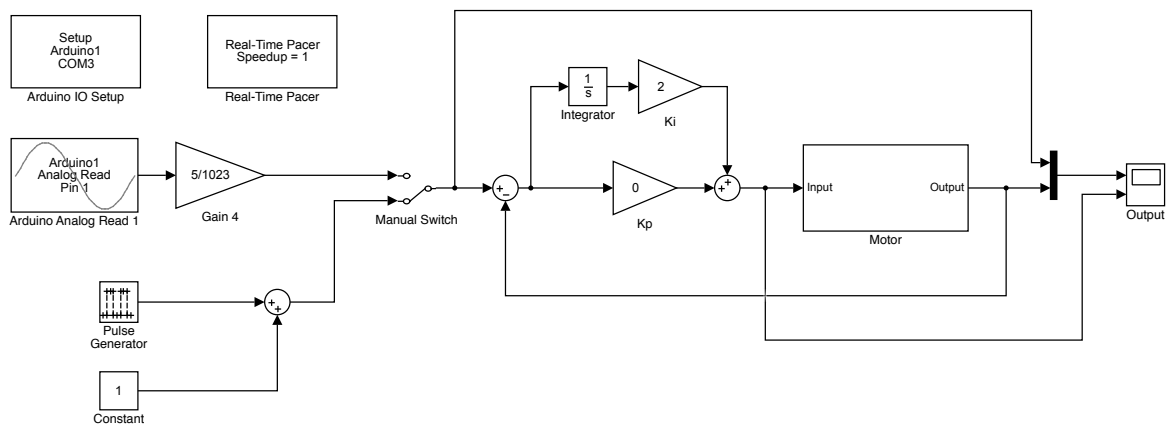
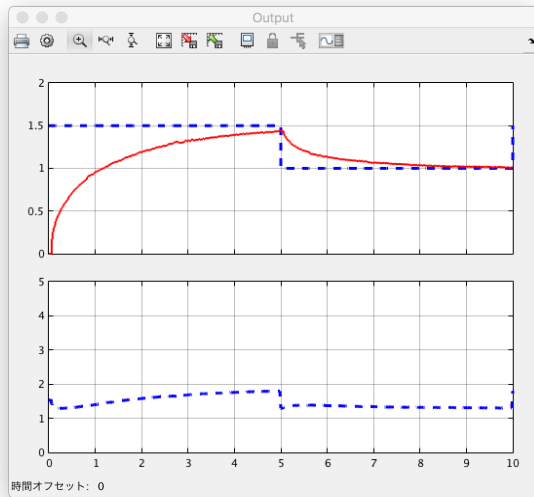
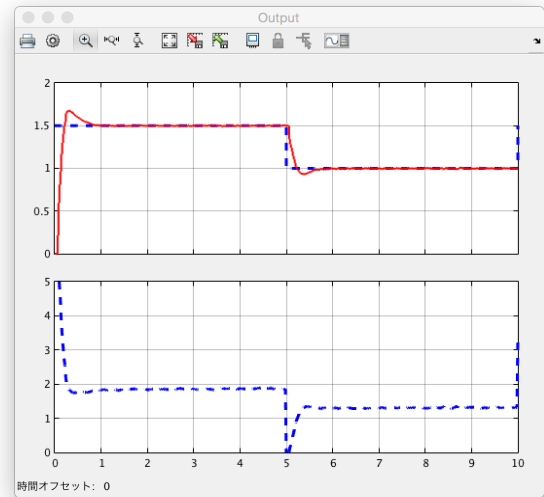


図 13: 比例積分制御系の Simulink モデル



(a) ある K_P, K_I の場合



(b) (a) とは別の K_P, K_I の場合

図 14: PI 制御系の実験結果

3.5 PI 制御

比例制御，積分制御の特徴をおよそ把握したところで，両ゲインを同時調整する **PI 制御**を試みる．積分ゲイン K_I を大きくして振動的な応答が得られているときに，比例制御を少し加えると振動を抑えられることが知られている．具体的なゲインの調整方法は以下のとおりである：

1. $K_P = 0$ のまま，多少振動的な出力が得られるまで K_I を徐々に大きくする．
2. K_I を固定したまま， K_P を少しずつ大きくする．振動が徐々に収まるので，適当なところで調整を終える．
3. K_I と K_P を微調整して，最終的な値を得る．

図 14 に実験結果の例を 2 通り示した．

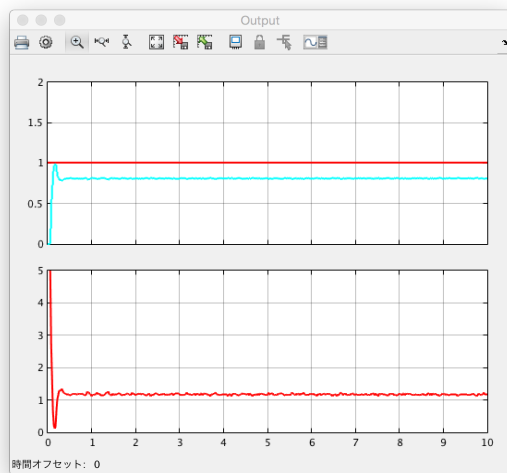
3.6 限界感度法による PI ゲイン決定

比例制御や積分制御のようにゲインが一つしかない場合と比べ，PI 制御のようにゲインが二つ以上になると，その調整にかかる労力は急増する．^{ジューグラー} Ziegler と ^{ニコルス} Nichols によって 1942 年に提案された限界感度法は，P 制御, PI 制御, PID 制御のゲイン調整法としてよく知られている．その手順は以下のとおりである：

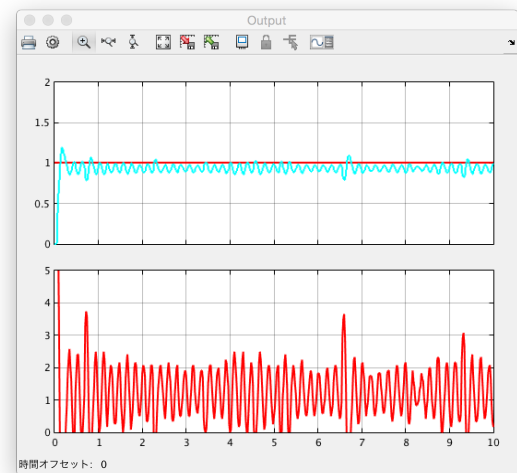
1. 比例制御において，出力が持続振動するまで比例ゲインを徐々に大きくし，持続振動時の値を K_c とおく．
2. 持続振動の振動周期を調べ，これを T_c とする．
3. 表 5 から，比例ゲイン K_P ，積分時間 T_I ，および微分時間 T_D を求める．

表 5: 限界感度法によるパラメータの決定

	比例ゲイン K_P	積分時間 T_I	微分時間 T_D
P 制御	$0.5K_c$	—	—
PI 制御	$0.45K_c$	$T_c/1.2$	—
PID 制御	$0.6K_c$	$T_c/2$	$T_c/8$



(a) ある K_P の場合



(b) 持続振動の状態に達した例

図 15: ステップ指令 1 V に対する P 制御（限界感度法によるゲイン調整のための予備実験）

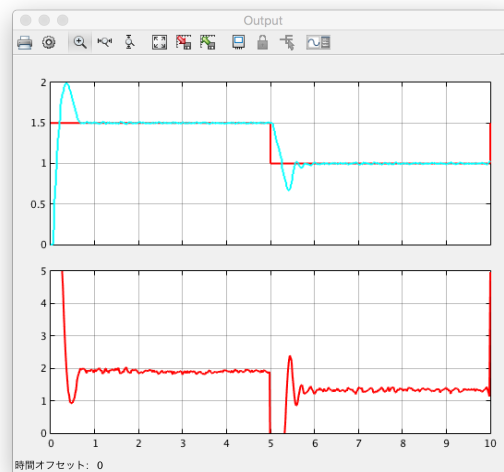


図 16: 限界感度法で求めた PI ゲインによる応答

4. $K_I = K_P/T_I$, $K_D = K_P T_D$ を用いて、積分ゲイン K_I や微分ゲイン K_D を求める。
5. 求めたゲインを用いて実験を行い、応答がさらに良好になるように各ゲインを微調整する。

限界感度法を本実験の PI 制御系 `velo_pi.mdl` に適用する場合、具体的な作業は以下の流れとなる：

1. Simulink モデル `velo_pi.mdl` において、[Manual Switch] をダブルクリックし [Pulse Generator] 側から [Arduino Analog Read 1] 側へ切り替える（一定目標値をポテンショメータから入力できるようにする）。
2. 比例ゲインを適当に、積分ゲインをゼロにして一度実験を行い、実験中に目標値が 1 V になるようポテンショメータを調整する（図 15a）。
3. 実験を繰り返しながら、出力が振幅がほぼ一定した持続振動の状態（図 15b）になるまで比例ゲインを大きくする。持続振動になったときの比例ゲインを K_c とする。
4. 10 秒間の出力のピーク数を数えるなどして、持続振動の振動周期 T_c を算出する（例：10 秒間に 50 周期の振動があるなら $T_c = 10/50 = 0.2$ ）。
5. 表 5 の PI 制御の行から、比例ゲイン K_P 、積分時間 T_I を求める。

6. 積分ゲインを $K_I = K_P/T_I$ から求める。
7. 求めたゲインを用いて制御を行い、応答がさらに良好になるように各ゲインを微調整する。
8. [Manual Switch] をダブルクリックし [Arduino Analog Read 1] 側から [Pulse Generator] 側へ切り替える。

限界感度法で得られた PI ゲインによる実験結果の例を図 16 に示す。

課題 3 PI 制御のゲインを 2 つの方法で調整した。調整の経験や実験結果をもとに両方法の相違点を説明せよ。

課題 4 [自由] 制御入力 $0 \sim 5 \text{ V}$ の間におさまらず、一時的に^{*)}飽和する場合、出力にはどのような影響が出ているか。意図的に作り出した、適当な実験結果を複数をもとに考察せよ。

課題 5 [自由] 上記 2 つ以外の PID ゲイン調整法について調べよ。また、そのうち 1 つの方法でゲインを調整し、3 つの方法の相違点を説明せよ。

4 モータの速度制御実験（モデルを用いる場合）

3 節では、実験を繰り返しながら PI ゲインを模索したり、限界感度法でさえ持続振動を起こす比例ゲインを見つけたりと試行錯誤を重ねるために多大な時間を要した。さらに、最終的な設計結果の妥当性を評価することが難しい。

このような試行錯誤のアプローチに対して、制御対象の特性を数式表現したモデルで表し、それに基づいて制御系の設計を行うモデルベース設計というアプローチがある。モデルベース設計には以下のようなメリットがある：

- ・ 実験に基づく試行錯誤が不要となり、要求仕様を満たす制御系が設計しやすくなる。
- ・ シミュレーションや解析が容易となり、時間的にも金銭的にもコストを抑えることができる。
- ・ 開発チーム内でモデルを共通言語とすることによって、連携強化、情報共有の容易化、問題点の早期発見などが期待できる。

モデルベースの制御系設計における本質的プロセスはモデリングである。実機の代わりとなるモデルの信頼性は非常に重要であり、モデルは実機の特性をよく表していなければならない。

4.1 モータのモデリング

本小節では制御対象であるモータのモデリングを行う。以降、Step 1) 物理法則をもとに数式モデルを導き、Step 2) 適切な近似で簡略化したうえで設計用のモデルを伝達関数として求める。

Step 1) 物理法則に基づく数式モデルの導出

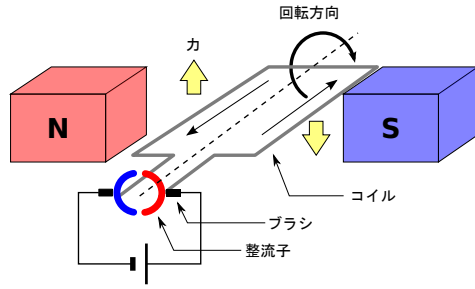
まず、物理法則をもとに、回路の動特性や回転体の動力学を微分方程式や代数方程式によって表現する。

一般的な小型 DC モータは、図 17a のように、固定子と呼ばれる磁石の間に電機子と呼ばれるコイルが置かれる。電機子に流れる電流の向きが常に同じとなるようにブラシと整流子とを設けると、電磁力が常に同じ向きに生じて電機子は回転する。この動作を抽象化し、電気・機械の基本要素で構成すると図 17b のように表現できる。図 17b において、 R_a は巻き線抵抗、 L_a は巻き線のインダクタンスを表し、モータへ電圧 $v_a(t)$ を加えると電機子に電流 $i_a(t)$ が流れる。一方、モータは回転すると、それにとまってモータの回転速度に比例する電圧（逆起電力）を生じる。逆起電力は、逆起電力定数と呼ばれる比例定数 K_e 、電機子の角速度 $\omega(t)$ を用いて、 $K_e \omega(t)$ と表すことができる。

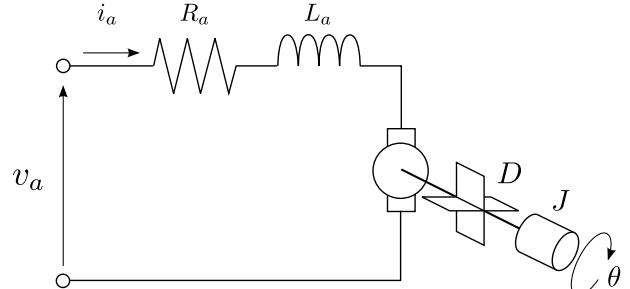
抵抗での電圧降下 $R_a i_a(t)$ 、コイルでの電圧降下 $L_a (di_a/dt)$ 、逆起電力 $K_e \omega(t)$ という 3 要素の和が、モータへの印加電圧 $v_a(t)$ とつり合うことから、つぎの回路方程式が得られる：

$$R_a i_a(t) + L_a \frac{di_a}{dt} + K_e \omega(t) = v_a(t). \quad (3)$$

^{*)} たとえば、5 s のステップのなかで 0.1~0.5 s 程度。



(a) 動作原理



(b) モデル図

図 17: DC モータ

(3) 式に従って電機子に電流 $i_a(t)$ が流れると、電機子を回転させるトルク（力）が発生する．発生するトルク $\tau(t)$ は電流に比例するので、次式で表現できる：

$$\tau(t) = K_t i_a(t). \quad (4)$$

ここで、 K_t はトルク定数と呼ばれる比例定数である．一方、慣性モーメント J 、粘性摩擦係数 D の回転体にトルク $\tau(t)$ が加わったときの運動方程式は

$$J \frac{d\omega}{dt} + D \omega(t) = \tau(t) \quad (5)$$

となる．さらに、電機子の回転角を $\theta(t)$ と表すと回転角の時間変化が角速度になるので、次式が成立する：

$$\frac{d\theta}{dt} = \omega(t). \quad (6)$$

各変数の初期値をゼロ ($i_a(0) = 0$, $\omega(0) = 0$, $\theta(0) = 0$) として (3)～(6) 式をラプラス変換すると、それぞれの s 領域での表現が以下のように求まる：

$$I_a(s) = \frac{1}{L_a s + R_a} (V_a(s) - K_e \Omega(s)), \quad (7)$$

$$\mathcal{T}(s) = K_t I_a(s), \quad (8)$$

$$\Omega(s) = \frac{1}{J s + D} \mathcal{T}(s), \quad (9)$$

$$\Theta(s) = \frac{1}{s} \Omega(s). \quad (10)$$

ただし、 $I_a(s) := \mathcal{L}[i_a(t)]$, $V_a(s) := \mathcal{L}[v_a(t)]$, $\Omega(s) := \mathcal{L}[\omega(t)]$, $\mathcal{T}(s) := \mathcal{L}[\tau(t)]$, $\Theta(s) := \mathcal{L}[\theta(t)]$ である．

Step 2) 設計用モデルの導出

入力 $u(t)$ に対して何らかの因果関係をもって出力 $y(t)$ が決まるものをシステムという．特に、線形性を満たし、特性が時間に依存して変わらないものを線形時不変システムと呼ぶ．線形時不変システムをラプラス変換した s 領域において、その入出力関係 $Y(s)/U(s)$ を表現したものが伝達関数である．伝達関数は線形時不変システムの性質を解析する場合や制御系を設計する場合によく用いられる．DC モータの場合、(7)～(10) 式より $I_a(s)$, $\mathcal{T}(s)$, $\Omega(s)$ を消去すると、電圧 $V_a(s)$ から回転角 $\Theta(s)$ までの伝達関数は次式として得られる：

$$\frac{\Theta(s)}{V_a(s)} = \frac{K_t}{s \{ (J s + D)(L_a s + R_a) + K_t K_e \}}. \quad (11)$$

予習 4 (3)～(6) 式をラプラス変換すると、 s 領域での表現として (7)～(10) 式が得られることを示せ．さらに、(7)～(10) 式より、電圧 $V_a(s)$ から回転角 $\Theta(s)$ までの伝達関数 (11) が求まることを示せ．ただし、導出過程を省略せず、数式の展開のみ示さず、文章を基本として丁寧に説明すること．

伝達関数 (11) は得られたがやや複雑な表現であるため、つづいて近似・簡略化することを考える。一般の小型 DC モータでは、電機子の応答である機械系の応答速度に比べて、回路方程式で記述される電気系の応答速度が十分速い。よって、インダクタンス L_a の影響は無視できると考えて問題ない。(11) 式において、 $L_a = 0$ とすると

$$\frac{\Theta(s)}{V_a(s)} \approx \frac{K_t}{s\{(Js + D)R_a + K_t K_e\}} = \frac{\frac{K_t}{R_a}}{s\left(Js + D + \frac{K_t K_e}{R_a}\right)} \quad (12)$$

を得る。 $\tilde{D} := D + K_t K_e / R_a$, $\tilde{K} := K_t / (\tilde{D} R_a)$, $T := J / \tilde{D}$ とおくと、(12) 式は

$$\frac{\Theta(s)}{V_a(s)} = \frac{\frac{K_t}{R_a}}{s(Js + \tilde{D})} = \frac{\frac{K_t}{\tilde{D} R_a}}{s\left(\frac{J}{\tilde{D}}s + 1\right)} = \frac{\tilde{K}}{s(Ts + 1)} \quad (13)$$

と表せる。(10) および (13) 式より、電圧 $V_a(s)$ から角速度 $\Omega(s)$ までの伝達関数は次式となる：

$$\frac{\Omega(s)}{V_a(s)} = \frac{\tilde{K}}{Ts + 1}. \quad (14)$$

さて、本実験の装置では、モータの回転速度を検出するために DC モータをタコメータとして用いている。つまり、実際に観測する出力はタコメータの電圧 $v_o(t)$ である。前述した電機子の角速度と逆起電力との関係と同様に、比例定数 K_o を用いて $\omega(t)$ と $v_o(t)$ との関係は時間領域で

$$v_o(t) = K_o \omega(t), \quad (15)$$

s 領域において

$$V_o(s) = K_o \Omega(s) \quad (16)$$

と記述できる。したがって、(14), (16) 式より、駆動用 DC モータへ印加する電圧（制御対象の入力） $U(s) := V_a(s)$ からタコメータの出力電圧（制御対象の出力） $Y(s) := V_o(s)$ までの伝達関数は次式で表現できる：

$$\frac{Y(s)}{U(s)} = \frac{K}{Ts + 1}. \quad (17)$$

ここで、 $K := K_o \tilde{K}$ である。(17) 式のパラメータ K はゲイン、 T は時定数と呼ばれ、それぞれの値は未知である。

4.2 ステップ応答法によるパラメータ同定

本小節では、未知パラメータである K と T の値を実験データから求める作業、パラメータ同定を行う。

(17) 式の形式で記述されるシステムのゲインと時定数を求める最も基本的な方法として、ステップ応答法がある。これは、入力に単位ステップ関数を加え、得られた出力の応答からゲイン K と時定数 T を直接読み取る方法である。

単位ステップ関数

$$u(t) = \begin{cases} 1, & t \geq 0 \\ 0, & t < 0 \end{cases} \quad (18)$$

はラプラス変換すると

$$U(s) = \frac{1}{s} \quad (19)$$

と表せる。制御対象の入出力関係は (17) 式より

$$Y(s) = \frac{K}{Ts + 1} U(s) \quad (20)$$

であり、(19) 式を作用させると

$$Y(s) = \frac{K}{Ts + 1} \cdot \frac{1}{s} = K \left(\frac{1}{s} - \frac{1}{s + \frac{1}{T}} \right) \quad (21)$$

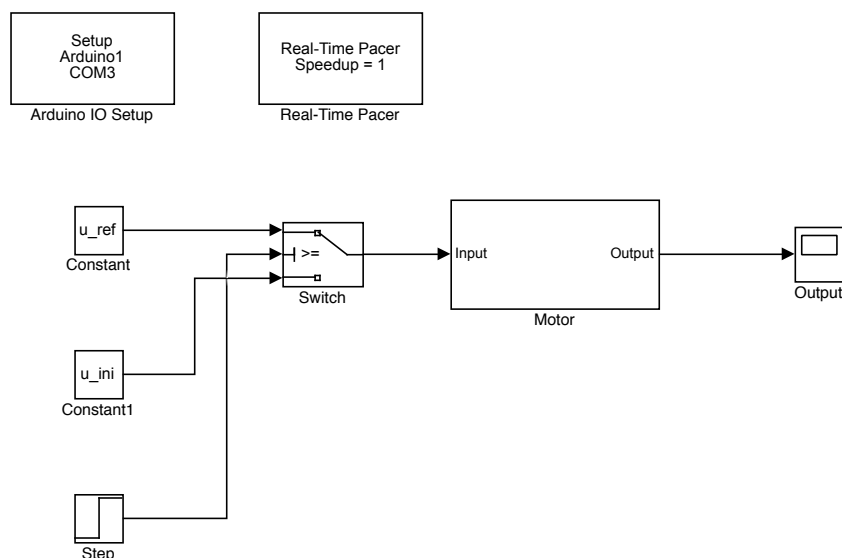


図 18: ゲイン同定（同定実験 1）のための Simulink モデル

となる．(21) 式を逆ラプラス変換することによって，出力の時間応答 $y(t)$ が次式として得られる：

$$y(t) = \mathcal{L}^{-1} \left[K \left(\frac{1}{s} - \frac{1}{s + \frac{1}{T}} \right) \right] = K(1 - e^{-\frac{t}{T}}). \quad (22)$$

時刻が十分経過する ($t \rightarrow \infty$) と $\lim_{t \rightarrow \infty} e^{-\frac{t}{T}} = 0$ となるため，出力の定常値 $y(\infty)$ は

$$y(\infty) = K \quad (23)$$

である．これは，出力の定常値として， K の値が実験データから読み取れることを意味する．一方， $t = T$ での $y(t)$ は

$$y(T) = K(1 - e^{-\frac{T}{T}}) = K(1 - e^{-1}) \approx 0.632K \quad (24)$$

であり，定常値の 63.2 % になるまでの時間を読み取ることによって定数 T の値が得られることを示している．

同定実験 1：ゲイン K の同定

まずシステムの定常特性からゲイン K のみ同定することを考える．

(23) 式から，定常状態におけるモータに入力する電圧 v_a とタコメータの出力電圧 v_o との関係は，次式で表せる：

$$v_o = K v_a. \quad (25)$$

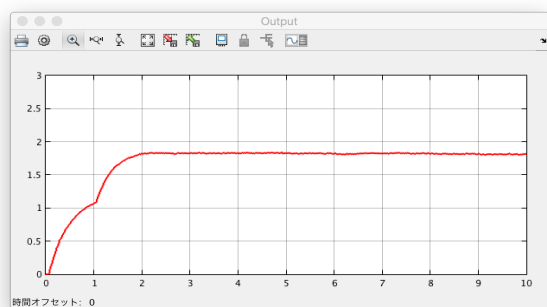
よって， v_a と v_o を実測して得られたデータに直線をあてはめれば，その傾きから K を同定できるはずである．

本実験では，MATLAB スクリプト `velo_id_gain.m` (プログラム 1) および Simulink モデル `velo_id_gain_sl.mdl` (図 18) を用いる．プログラム 1 によって，モータへ入力する電圧 v_a を 1 V から 2.5 V まで，0.125 V 刻みで変化させながら，タコメータの出力電圧 v_o を記録していくことができる．具体的な実験手順を以下に示す：

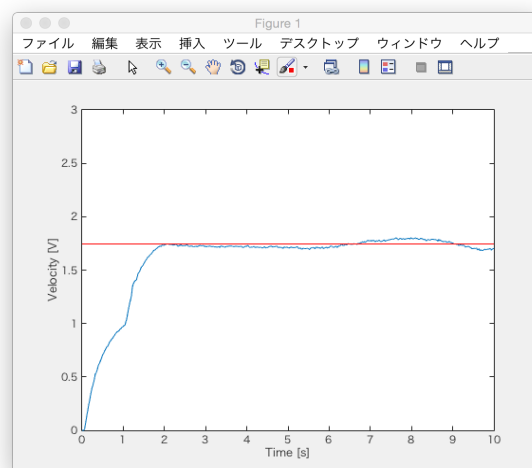
1. MATLAB コマンドラインで以下を実行する．実験中，図 19 のような表示が v_a の値ごとに得られる．

» `velo_id_gain` ↵

2. v_a 全パターンの実験が終了すると，MATLAB コマンドラインにて `Umin` と `Umax` の値を尋ねてくる (図 20a)．
図 21a を観察し，直線のあてはめに適当そうな横軸の範囲を検討して `Umin` と `Umax` の値を与える (図 20b)．
3. K と `u_offset` の値が得られる (図 21b)．直線近似が適当でなければ，2 (Retry) を選択して範囲を指定し直す．



(a) Scope



(b) Figure 1

図 19: 実験中の表示例（同定実験 1）

```

MATLAB R2015a - academic use
ホーム プロ... アプリケ...
« lec+exp » app-exp » new-theme » contr-design8arduino+matlab »
No operation will be performed on the Arduino board
Connection to 'Arduino1' successfully closed
velo_id_gain_sl/Real-Time Pacer: Average idle real time per major time step = 0.
y_mean = 1.687439
Attempting connection .....
Basic Analog and Digital I/O (adio.pde) sketch detected !
Arduino successfully connected !
Warning: the sketch running on the Arduino does not support the motor shield
No operation will be performed on the Arduino board
Warning: the sketch running on the Arduino does not support the motor shield
No operation will be performed on the Arduino board
Warning: the sketch running on the Arduino does not support the motor shield
No operation will be performed on the Arduino board
Warning: the sketch running on the Arduino does not support the motor shield
No operation will be performed on the Arduino board
Warning: the sketch running on the Arduino does not support the motor shield
No operation will be performed on the Arduino board
Warning: the sketch running on the Arduino does not support the motor shield
No operation will be performed on the Arduino board
Connection to 'Arduino1' successfully closed
velo_id_gain_sl/Real-Time Pacer: Average idle real time per major time step = 0.
y_mean = 1.745198
Please input Umin and Umax for fit
Umin =
入力待ち

```

(a) Umin, Umax 入力前

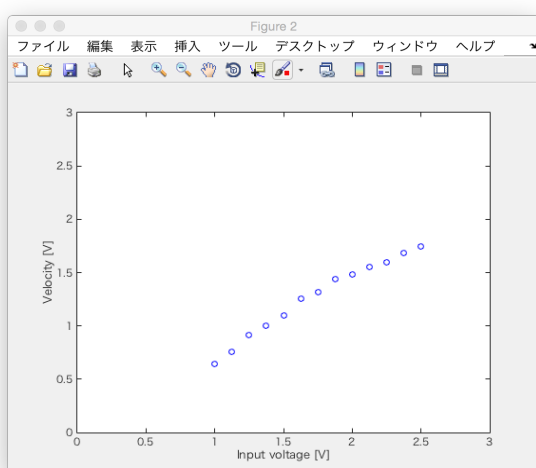
```

MATLAB R2015a - academic use
ホーム プロ... アプリケ...
« lec+exp » app-exp » new-theme » contr-design8arduino+matlab »
No operation will be performed on the Arduino board
Warning: the sketch running on the Arduino does not support the motor shield
No operation will be performed on the Arduino board
Warning: the sketch running on the Arduino does not support the motor shield
No operation will be performed on the Arduino board
Warning: the sketch running on the Arduino does not support the motor shield
No operation will be performed on the Arduino board
Warning: the sketch running on the Arduino does not support the motor shield
No operation will be performed on the Arduino board
Warning: the sketch running on the Arduino does not support the motor shield
No operation will be performed on the Arduino board
Connection to 'Arduino1' successfully closed
velo_id_gain_sl/Real-Time Pacer: Average idle real time per major time step = 0.
y_mean = 1.745198
Please input Umin and Umax for fit
Umin = 1
Umax = 2.5
*** Motor parameters ***
K = 0.731616
u_offset = 0.017480
OK? (1:Quit, 2:Retry) = 2
Please input Umin and Umax for fit
Umin = 1
Umax = 2
*** Motor parameters ***
K = 0.865216
u_offset = 0.229865
OK? (1:Quit, 2:Retry) =
入力待ち

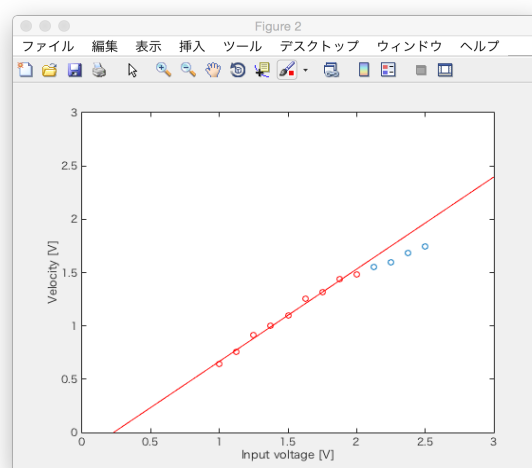
```

(b) Umin, Umax 入力後

図 20: MATLAB コマンドラインの様子（同定実験 1）



(a) 実験データ



(b) 近似曲線のあてはめ

図 21: モータへの入力電圧とタコメータの出力電圧との関係（同定実験 1）

課題6 同定実験1で得られた結果に対して、(25)式へのあてはまり具合を確認すると、何らかの誤差を観察できるはずである。どの程度の誤差か説明し、その誤差が生じた物理的要因を多角的に検討して洞察せよ。

プログラム 1: velo_id_gain.m

```
1 %% velo_id_gain.m
2
3 %% Initialize
4 clear all
5 close all
6
7 %% Parameters
8 ts = 1/50;
9 u_ini = 1.5;
10
11 %% Define input voltage list
12 u_ref_list = 1:0.125:2.5;
13
14 %% Open simulink model
15 open_system('velo_id_gain_sl');
16 open_system('velo_id_gain_sl/Output');
17
18 %% Start experiment
19 y_mean_list = [];
20 for i=1:length(u_ref_list)
21     u_ref = u_ref_list(i);
22     sim('velo_id_gain_sl');
23     t = yout.time;
24     y = yout.signals.values;
25     y_mean = mean(y(250:end));
26     fprintf('y_mean_=%f\n', y_mean);
27     y_mean_list(i) = y_mean;
28     % Plot figure
29     figure(1)
30     plot(t, y, t, ones(size(y))*y_mean, 'r');
31     xlabel('Time_[s]', ylabel('Velocity_[V]'))
32     axis([0 10 0 3])
33     drawnow
34 end
35
36 %% Plot data
37 figure(2)
38 plot(u_ref_list, y_mean_list, 'bo')
39 xlabel('Input_voltage_[V]')
40 ylabel('Velocity_[V]')
41 axis([0 3 0 3])
42
43 %% Calculate parameters
44 while(1)
45     disp('Please_input_Umin_and_Umax_for_fit')
46     umin = input('Umin_');
47     umax = input('Umax_');
48     sidx = min(find(u_ref_list >= umin));
49     eidx = max(find(u_ref_list <= umax));
50     P = polyfit(u_ref_list(sidx:eidx), y_mean_list(sidx:eidx), 1);
51     Pin = 0:3;
52     Pout = P(1)*Pin + P(2);
53     u_offset = -P(2)/P(1);
54     fprintf('***_Motor_paramters_***\n')
55     fprintf('K_=%f\n', P(1));
56     fprintf('u_offset_=%f\n', u_offset);
57
```

```

58 figure(2)
59 plot(u_ref_list,y_mean_list,'o',...
60      u_ref_list(sidx:eidx),y_mean_list(sidx:eidx),'ro',...
61      Pin,Pout,'r-')
62 xlabel('Input_voltage_[V]')
63 ylabel('Velocity_[V]')
64 axis([0 3 0 3])
65
66 sw = input('OK?(1:Quit,2:Retry)=');
67 switch sw
68     case 1;
69         break;
70     case 2;
71         continue;
72 end
73 end
74
75 %% EOF of velo_id_gain.m

```

同定実験 2：ゲイン K と時定数 T の同定

同定実験 1 で得た知見をもとに、ステップ応答法を工夫して適用し、ゲイン K と時定数 T の同定を試みる。

本実験では、MATLAB スクリプト `velo_id_tc.m`（プログラム 2）および Simulink モデル `velo_id_tc_sl.mdl`（図 22）を用いる。具体的な実験手順は以下のとおりである：

1. MATLAB コマンドラインで以下を実行する。

```
» velo_id_tc ↵
```

2. 実験中、図 23 のような表示が得られ、完了すると図 24, 25 が結果として得られる。図 25b は、図 25a の 10～14 秒のデータであるが、ステップ応答が 0 から始まるよう縦軸の値を調整していることに注意されたい。

予習 5 同定実験 2 ではどのようなステップ入力を与え、ステップ応答法を適用しているか。プログラム 2 および図 22 から読み取り、ステップ入力の概形を図示して説明せよ。

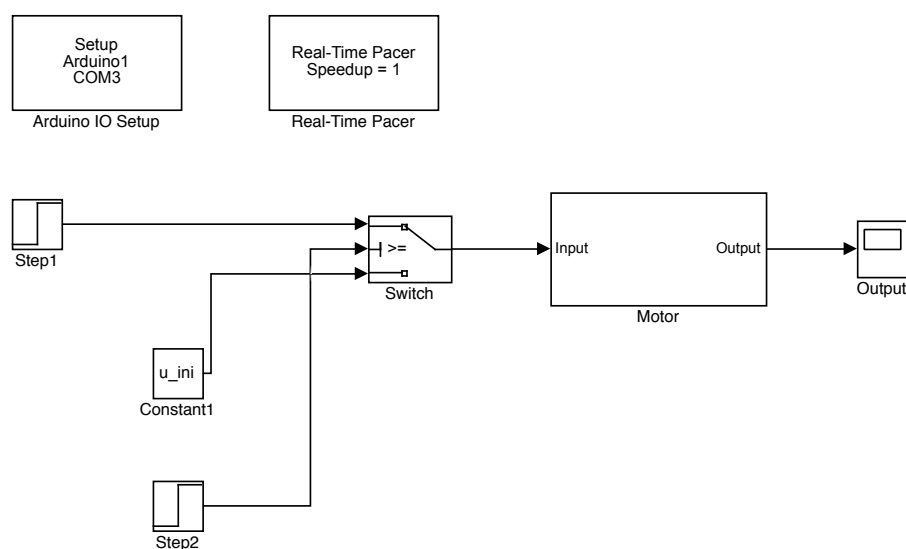


図 22: ステップ応答法（同定実験 2）のための Simulink モデル

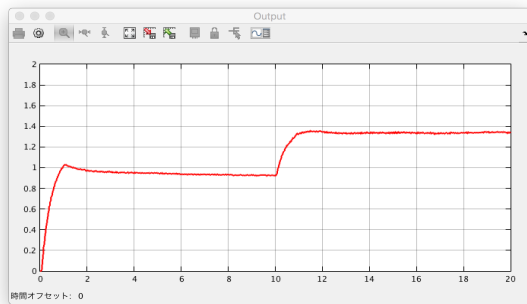
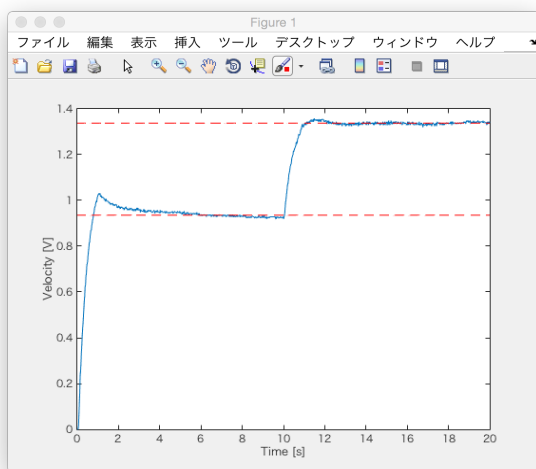


図 23: Scope の様子 (同定実験 2)

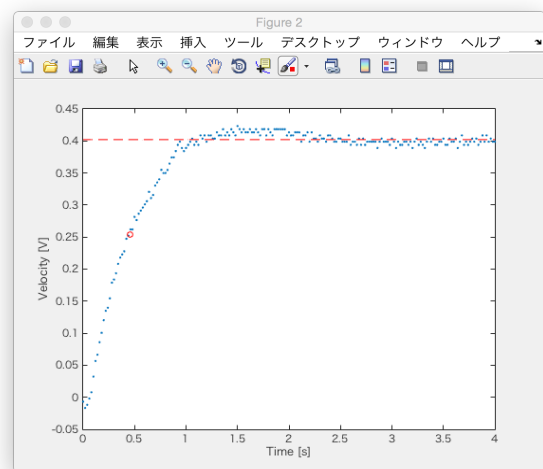
```
MATLAB R2015a - academic use
ホーム プロ... アプリケ...
<< lec+exp >> app-exp >> new-theme >> contr-design&arduino+matlab >
ワークスペース 現在のフォルダー

>> velo_id_tc
Attempting connection .....
Basic Analog and Digital I/O (adio.pde) sketch detected !
Arduino successfully connected !
Warning: the sketch running on the Arduino does not support the motor shield
No operation will be performed on the Arduino board
Warning: the sketch running on the Arduino does not support the motor shield
No operation will be performed on the Arduino board
Warning: the sketch running on the Arduino does not support the motor shield
No operation will be performed on the Arduino board
Warning: the sketch running on the Arduino does not support the motor shield
No operation will be performed on the Arduino board
Warning: the sketch running on the Arduino does not support the motor shield
No operation will be performed on the Arduino board
Warning: the sketch running on the Arduino does not support the motor shield
No operation will be performed on the Arduino board
Connection to 'Arduino1' successfully closed
velo_id_tc_sl/Real-Time Pacer: Average idle real time per major time step = 0.00
== Results ==
K      = 0.804205
T      = 0.460000
u_offset = 0.136748
f>>
```

図 24: MATLAB コマンドラインの様子 (同定実験 2)



(a) 取得したデータ全体



(b) 10～14 秒におけるデータ (時定数の計算用)

図 25: ステップ応答 (同定実験 2)

プログラム 2: velo_id_tc.m

```
1 %% velo_id_tc.m
2
3 %% Initialize
4 clear all
5 close all
6
7 %% Parameters
8 ts      = 1/50;
9 u_ini   = 1.5; % initial input
10 r_const = 1.3; % offset voltage
11 p_const = 0.5; % step input voltage
12 s_time  = 10; % step time
13 w_time  = 4;  % wait time
14
15 %% Open simulink model
16 open_system('velo_id_tc_sl');
17 open_system('velo_id_tc_sl/Output');
18
19 %% Start experiment
20 sim('velo_id_tc_sl');
```

```

21
22 %% Parameter identification
23 y = yout.signals.values;
24 t = yout.time;
25
26 c1 = mean(y(w_time/ts:s_time/ts));
27 c2 = mean(y((s_time+w_time)/ts:end));
28
29 figure(1)
30 plot(t,y,...
31      t,ones(size(t))*c1,'r--',...
32      t,ones(size(t))*c2,'r--')
33 xlabel('Time_[s]')
34 ylabel('Velocity_[V]')
35
36 %% Calculate Gain
37 K_id = (c2-c1)/p_const;
38 u_offset = (K_id*r_const -c1)/K_id;
39
40 %% Calculate time constant
41 y2 = y(s_time/ts:end) - c1;
42 t2 = t(s_time/ts:end);
43 t2 = t2 - t2(1);
44
45 tc_idx = min(find(y2 > (c2-c1)*0.632));
46 T_id = t2(tc_idx);
47
48 figure(2)
49 plot(t2,y2,'.',...
50      T_id,(c2-c1)*0.632,'ro',...
51      t2,ones(size(t2))*(c2-c1),'r--')
52 xlim([0 w_time])
53 xlabel('Time_[s]'), ylabel('Velocity_[V]')
54
55 %% Display results
56 fprintf('==_Results_==\n')
57 fprintf('K_====_%f\n',K_id)
58 fprintf('T_====_%f\n',T_id)
59 fprintf('u_offset_=%f\n',u_offset)
60
61 %% EOF of velo_id_tc.m

```

検証実験：同定結果の検証

同定実験 2 で求めたパラメータを使って、モデルの応答と実機の応答とがどの程度一致するか検証する。本実験では、MATLAB スクリプト `velo_id_verify.m`（プログラム 3）および Simulink モデル `velo_id_verify_sl.mdl`（図 26）を用いる。具体的な実験手順を以下に示す：

1. `velo_id_verify.m` 内のパラメータ K , T , u_offset を、各自が同定実験 2 で得た値に書き換える。
2. `velo_id_verify_sl.mdl` を開き、[Output] をダブルクリックして実験中に応答が表示されるようにする。
3. MATLAB コマンドラインで以下を実行する。

```
» velo_id_verify
```

4. 図 27 のような結果が得られる。ノイズを含むほう（実線）が実機の応答、そうでないほう（破線）がモデルの応答である。

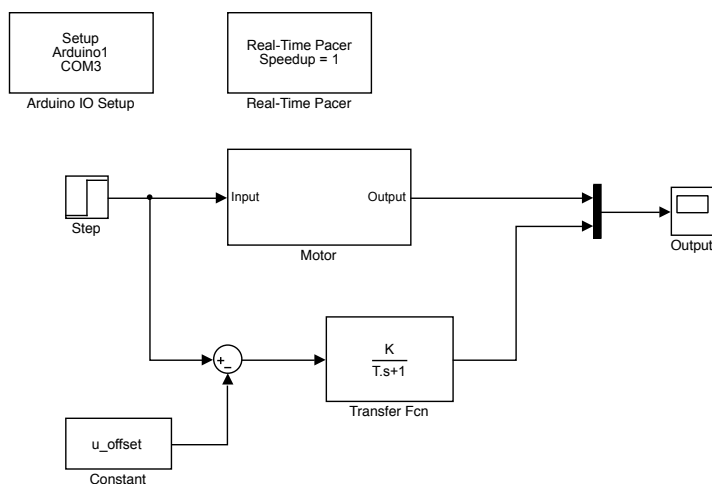


図 26: 同定結果検証のための Simulink モデル

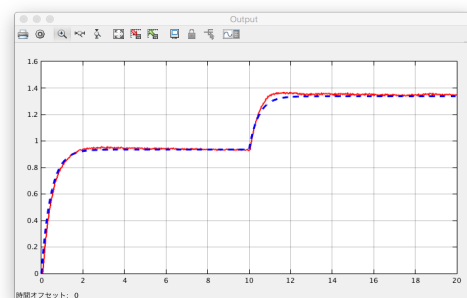


図 27: 実験結果の一例（検証実験）

プログラム 3: velo_id_verify.m

```

1 %% velo_id_verify.m
2
3 %% Set identified parameters
4 K      = 1.57;
5 T      = 0.76;
6 u_offset = 1.03;
7
8 %% Open simulink model
9 open_system('velo_id_verify_sl');
10 open_system('velo_id_verify_sl/Output');
11
12 %% Start experiment
13 sim('velo_id_verify_sl')
14
15 %% Save Parameters
16 save model_data K T u_offset
17
18 %% EOF of velo_id_verify.m

```

4.3 極指定法による PI ゲイン設計

4.1 節で制御対象のモデルを導き、4.2 節で未知パラメータを同定した。本小節では、所望の特性となるようモデルを用いて PI ゲインを設計する。

図 28 に示すフィードバック制御系において、制御対象の伝達関数を

$$P(s) := \frac{K}{Ts + 1}, \quad (26)$$

PI 制御器の伝達関数を

$$C(s) := K_P + \frac{K_I}{s} \quad (27)$$

とする。目標値 $R(s)$ から出力 $Y(s)$ までの伝達関数は、

$$Y(s) = P(s)U(s), \quad (28)$$

$$U(s) = C(s)E(s), \quad (29)$$

$$E(s) = R(s) - Y(s) \quad (30)$$

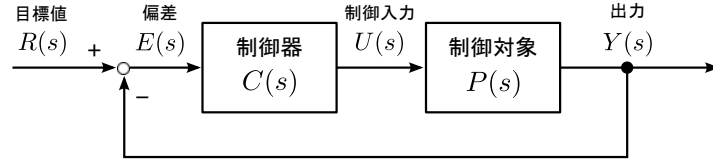


図 28: フィードバック制御系

より $U(s)$, $E(s)$ を消去すると

$$Y(s) = P(s)C(s)(R(s) - Y(s)) \Leftrightarrow \frac{Y(s)}{R(s)} = \frac{P(s)C(s)}{1 + P(s)C(s)} \quad (31)$$

と得られる。(31) 式右辺を具体的に求めると、次式のようになる：

$$\frac{Y(s)}{R(s)} = \frac{\frac{K}{Ts+1} \left(K_P + \frac{K_I}{s} \right)}{1 + \frac{K}{Ts+1} \left(K_P + \frac{K_I}{s} \right)} = \frac{\frac{KK_P}{T}s + \frac{KK_I}{T}}{s^2 + \frac{1+KK_P}{T}s + \frac{KK_I}{T}}. \quad (32)$$

伝達関数の分母をゼロと等しいとした方程式は特性方程式と呼ばれ、線形時不変システムの応答は特性方程式の根（これを極という）と対応することが知られている。そこで、フィードバック制御系の伝達関数の極が望みの配置となるよう PI ゲインを決定することを考える。この方法は極指定法と呼ばれている。

(32) 式の特性方程式は (32) 式より

$$s^2 + \frac{1+KK_P}{T}s + \frac{KK_I}{T} = 0 \quad (33)$$

である。一方、望みの極を p_1, p_2 とすると、これらを根とする方程式は

$$(s - p_1)(s - p_2) = s^2 - (p_1 + p_2)s + p_1p_2 = 0 \quad (34)$$

と表せる。(33), (34) 式の係数を比較すると、

$$\begin{cases} \frac{1+KK_P}{T} = -(p_1 + p_2), \\ \frac{KK_I}{T} = p_1p_2 \end{cases} \quad (35)$$

を満たす PI ゲインが次式のように求まる：

$$K_P = -\frac{(p_1 + p_2)T + 1}{K}, \quad K_I = \frac{p_1p_2T}{K}. \quad (36)$$

極指定法の実験

本実験では、MATLAB スクリプト `velo_pi_mbd.m`（プログラム 4）および Simulink モデル `velo_pi_mbd_sl.mdl`（図 29）を用いる。`velo_pi_mbd_sl.mdl` には 2 つのフィードバックループが存在するが、上側は実験装置に対するフィードバック制御系、下側は同定したモデルに対するフィードバック制御系である。[Saturation] ブロックや [Offset] ブロックがあるのは、実験装置が入力電圧の制限（0～5 V の範囲内）や `u_offset` の存在を考慮するためである。`velo_pi_mbd.m` では、パラメータ同定した K, T, u_offset を `model_data.mat` から読み込んでいる。

プログラム 4: `velo_pi_mbd.m`

```
1 %% velo_pi_mbd.m
2
3 %% Initialize & load data
4 close all
5 clear all
6 load model_data
7
```



```

8 %% PI design by pole placement
9 p1 = input('p1=_');
10 p2 = p1; % 重根の場合
11 % p2 = input('p2 = '); % p2も指定する場合
12
13 Kp = -((p1+p2)*T + 1)/K;
14 Ki = p1*p2*T/K;
15
16 %% Display PI parameters
17 disp('>>>PI_parameters<<<')
18 fprintf('Kp=_\n',Kp);
19 fprintf('Ki=_\n',Ki);
20
21 %% Open simulink model
22 open_system('velo_pi_mbd_sl');
23 open_system('velo_pi_mbd_sl/Output');
24
25 %% Experiment
26 ts = 1/50;
27 sim('velo_pi_mbd_sl')
28
29 %% EOF of velo_pi_mbd.m

```

具体的な実験手順を以下に示す：

1. velo_pi_mbd_sl.mdl を開き，[Output] をダブルクリックして実験中に応答が表示されるようにしておく．
2. MATLAB コマンドラインで以下を実行する．

```
» velo_pi_mbd
```

3. 指定したい極を求めてくるため，適当な値を入力する（図 30）．なお，デフォルトでは $p_1 = p_2$ としているので， p_2 も指定したい場合はプログラム 4 の 10 行目をコメントアウトし，11 行目を有効にする．
4. 指定した極に応じて実験結果が図 31 のように得られる．

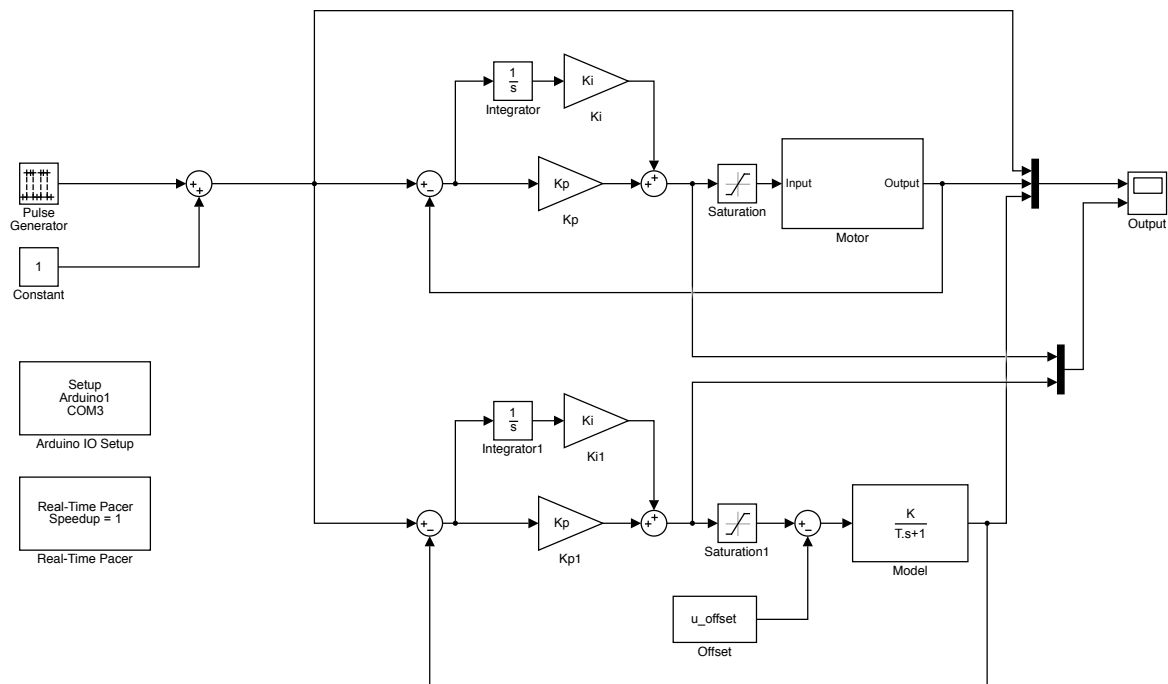


図 29: モデルベース設計による PI 制御系の Simulink モデル

- | | |
|-------|--|
| 課題 7 | $p_1 = p_2$ のもと、望みの極を実軸上で様々に変化させると、応答はどのように変化するか。実験結果をもとに、極と応答の間にどんな関係性があるか考察せよ。 |
| 課題 8 | p_1, p_2 を共役な複素数としたとき、望みの極を虚軸と平行な方向に様々に変化させると、応答はどのように変化するか。実験結果をもとに、極と応答の間にどんな関係性があるか考察せよ。 |
| 課題 9 | PI ゲインの設計問題に対して、モデルを用いないアプローチとモデルを用いるアプローチをとった。調整や設計における手間（時間的コスト）や得られた結果といった観点から、2つのアプローチを比較評価せよ。 |
| 課題 10 | [自由] モデルを用いないアプローチで得たゲインを極として評価し、極指定法で得た極と比較・議論せよ。 |

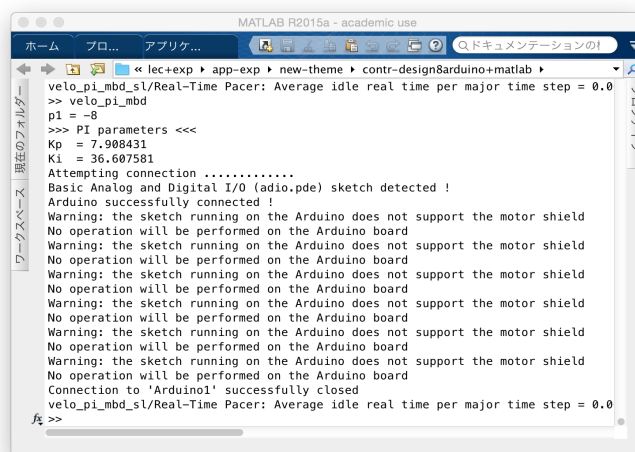
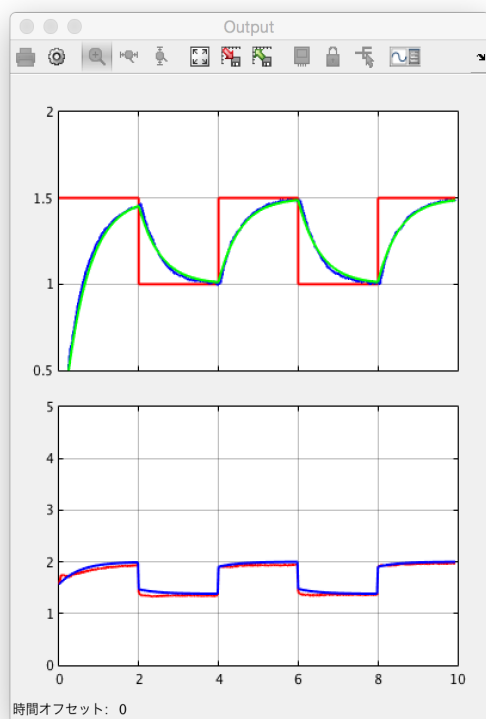
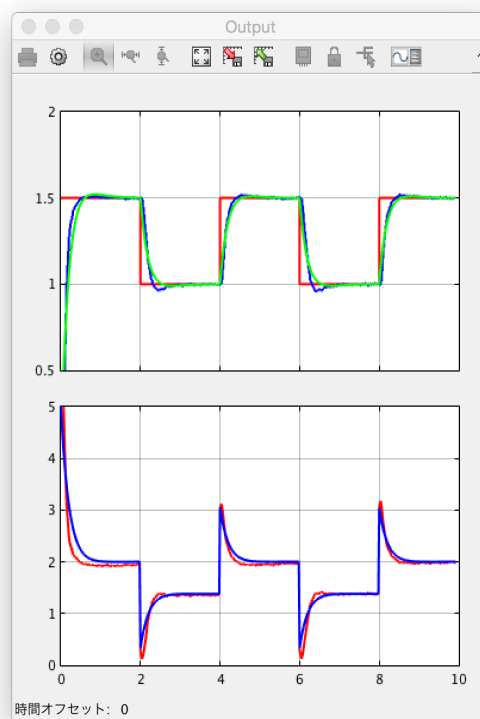


図 30: MATLAB コマンドラインの様子 (極指定法)



(a) ある $p_1 (= p_2)$ の場合



(b) (a) とは別の $p_1 (= p_2)$ の場合

図 31: 実験結果の例 (極指定法)

参考文献

- [1] 木下 是雄：“理科系の作文技術”，中公新書，1981。 ※レポートの書き方に関する古典的名著
- [2] 別府 俊幸・渡辺 賢治：“工学系卒論の書き方”，コロナ社，2020。 ※ YouTube 講義動画あり
- [3] 化学同人編集部：“実験データを正しく扱うために”，化学同人，2007。 ※実験データの扱い方に困ったら
- [4] 本多 勝一：“日本語の作文技術”，朝日文庫，1982。 ※読み手にわかりやすい文章を目指すなら
- [5] Arduino: “Introduction,” 2018. Accessed: 29 Mar 2021. [Online] <https://www.arduino.cc/en/Guide/Introduction>
- [6] 小林 茂：“フィジカルコンピューティング概論”，特集「夏休み工作のためのフィジカルコンピューティング」，情報処理，Vol.52, No.8, pp.914–917, 2011.
- [7] ITP Physical Computing: “What is Physical Computing,” 2014. Accessed: 29 Mar 2021. [Online] <https://itp.nyu.edu/physcomp/>
- [8] 平田 光男：“Arduino と MATLAB で制御系設計をはじめよう！”，TechShare 株式会社，2012。 ※全般参照

謝辞

予備実験の実施，実験キットの組み立て／整備を通して，本稿の執筆／改訂や学生実験の運営に貢献してくれた，大学院情報科学研究科博士前期課程 2016 年度生の安達 勇介さん，日下部 宏幸さん，情報科学部情報科学科 2016 年度生の鈴木 涼斗さん，中山 万由さん，2017 年度生の安藤 祐太さん，2018 年度生の加藤 伶奈さん，山口 空良さん，2019 年度生の柴田 真理菜さん，杉浦 功汰さん，各位の協力を感謝いたします。

付録 A Simulink でのデータ出力

実験指導書では実験中の様子にあわせ，Scope ブロックから得られるグラフを画面キャプチャして実験結果の例として示しているが，このような方法で得たグラフはレポートにはふさわしくない（1.4 節参照）。また，実験中，その都度グラフを作成することは作業効率がよいとはいえない。ここでは，実験データをデータファイルとして保存し，実験後そのデータをグラフ化する方法を簡潔に説明する。

A.1 To Workspace ブロックの利用からデータファイルへの保存

To Workspace ブロックを利用すれば，入力した信号を Workspace（MATLAB コマンドライン）に構造体変数として保存できる。

[Simulink ライブラリ ブラウザ] ウィンドウから，[Simulink]，[Sinks] とたどると To Workspace ブロックが見つかる。このブロックをドラッグ&ドロップ操作で，[Sinks] から作業中の Simulink モデルの適当な場所にコピーする。Scope ブロックと同じように Mux ブロックで複数の信号をまとめることができる。構造体変数名はデフォルトで `simout` であるが，（To Workspace ブロックをダブルクリックして開く）ブロックパラメータで変更することができる。

To Workspace ブロックを配置・接続後，一度シミュレーションを実行すると，MATLAB コマンドラインで

```
» simout.Time
```

とすれば時刻系列が，

```
» simout.Data(:,1)
```

とすれば第 1 データ系列が得られる（第 2 データ系列以降も同様）。ワークスペース変数をファイルへ保存する方法はいくつかあるが，`save` コマンドを用いるなら以下のようにコマンドを実行する：

```
» output = [simout.Time simout.Data];  
» save('sim.dat', 'output', '-ascii')
```

この結果、作業ディレクトリ内にスペース区切りのテキストデータファイル `sim.dat` が作成される。

A.2 データのグラフ化

スペース区切りのテキストデータファイルが扱える任意のデータ可視化ツールを用いて、得られたデータのグラフを生成する。以降では、GNU PLOT による方法と MATLAB による方法の 2 つを紹介する。

A.2.1 GNU PLOT による方法

ターミナルでつぎのコマンドを実行して GNU PLOT を起動する：

```
$ gnuplot
```

たとえば、第 2、第 3 データ系列の時間変化をグラフ表示する場合、以下のように実行すれば良い^{vi)}：

```
gnuplot> set grid  
gnuplot> set xlabel 'time (s)'  
gnuplot> set ylabel 'Input & Output (V)'  
gnuplot> plot 'sim.dat' u 1:2 t 'input' w l, 'sim.dat' u 1:3 t 'output' w l
```

表示したグラフを PDF 形式で保存したい場合は上記につづけて以下を実行する：

```
gnuplot> set term pdf  
gnuplot> set output 'sim.pdf'  
gnuplot> replot
```

以上の処理に慣れたら、効率的に処理するためにバッチ処理（スクリプト）化するとよい。情報科学部 wiki (<https://www.cis.aichi-pu.ac.jp/dokuwiki/doku.php?id=index.html>) の「計算機利用の手引き Web 版」の「gnuplot の利用方法」などを参照されたい。

A.2.2 MATLAB でデータファイルを読み込み、グラフを作成する方法

つぎのように実行すれば、`sim.dat` に保存されていたデータをワークスペース変数 `simdat` に読み込むことができる：

```
» simdat = load('sim.dat', '-ascii')
```

たとえば、第 2、第 3 データ系列の時間変化をグラフ表示する場合、以下のように実行すれば良い：

```
» plot(simdat(:,1), simdat(:,2:3))  
» xlabel('time (s)')  
» ylabel('Input & Output (V)')  
» legend('input', 'output', 'Location', 'northoutside', 'Orientation', 'horizontal')  
» grid on
```

以上のコマンド実行は M-ファイルという形式で、GNU PLOT と同じように、バッチ処理（スクリプト）化できる。「計算機利用の手引き」の第 11 章「MATLAB の利用方法」などを参照されたい。

^{vi)} GNU PLOT では多くのコマンド、オプションを略して表記することができる。ここでは、たとえば `plot` コマンドにおいて、`using 1:2` を `u 1:2`、`title 'input'` を `t 'input'`、`with lines` を `w l` と表記した。

A.3 グラフ作成の具体例

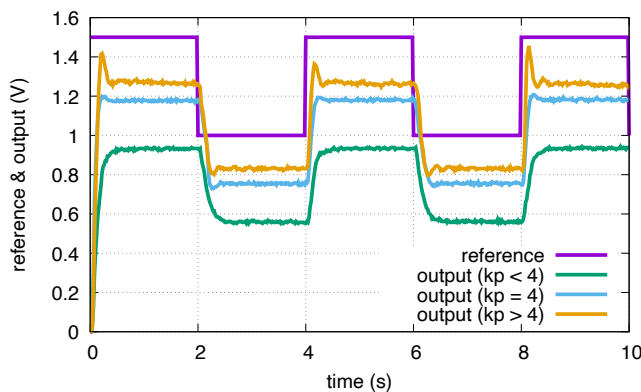
GNUPLLOT, MATLAB 各方法でレポート用グラフを作成した例を示す。図 32 は, 3 種類の K_p で P 制御した実験データ `velo_p_kpa.dat`, `velo_p_kpb.dat`, `velo_p_kpc.dat` があり, その応答の違いを比較するグラフである。GNUPLLOT でスクリプト 5 により作成したものが図 32a, MATLAB でスクリプト 6 により作成したものが図 32b である。

プログラム 5: `plot_pcontr.gp`

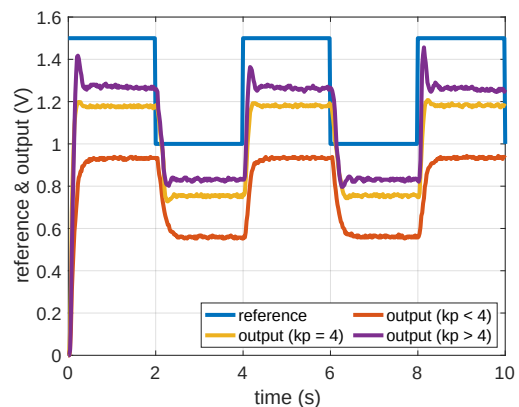
```
1 set term pdf solid color font 'Helvetica',16'
2
3 set key right bottom
4 set grid
5 set xlabel 'time_(s)'
6 set ylabel 'reference_\&output_(V)'
7 set output 'pcontr_comp8gp.pdf'
8 plot 'velo_p_kpa.dat' u 1:2 t 'reference' w l lw 4, \
9      'velo_p_kpa.dat' u 1:3 t 'output_(kp<4)' w l lw 4, \
10     'velo_p_kpb.dat' u 1:3 t 'output_(kp=4)' w l lw 4, \
11     'velo_p_kpc.dat' u 1:3 t 'output_(kp>4)' w l lw 4
```

プログラム 6: `plot_pcontr.m`

```
1 sim4kpa = load('velo_p_kpa.dat','-ascii');
2 sim4kpb = load('velo_p_kpb.dat','-ascii');
3 sim4kpc = load('velo_p_kpc.dat','-ascii');
4
5 figure
6 plot(sim4kpa(:,1),sim4kpa(:,2:3), 'Linewidth',4), hold on
7 plot(sim4kpb(:,1),sim4kpb(:,3), 'Linewidth',4)
8 plot(sim4kpc(:,1),sim4kpc(:,3), 'Linewidth',4)
9 ylim([0 1.6]), grid on
10 xlabel('time_(s)'), ylabel('reference_\&output_(V)')
11 legend('reference','output_(kp<4)',...
12       'output_(kp=4)','output_(kp>4)', ...
13       'Location','southeast', ...
14       'Orientation','horizontal', 'NumColumns',2)
15 set(findobj('type','axes'),'fontsize',18);
16
17 print('pcontr_comp8ml.pdf', '-dpdf')
```



(a) GNUPLLOT での作成例



(b) MATLAB での作成例

図 32: レポート用グラフの例 (P 制御における応答比較)

令和5年度 情報科学実験I データ科学

担当教員： 小畑建太

1 データ科学と実験目的

近年、コンピュータ関連機器の高性能化・低価格化や多種多様なデバイス・センサー等がインターネットに繋がった IoT (Internet of Things) 関連機器の普及により、リアルタイムまたは迅速な大規模データの転送、蓄積および解析が一般的に可能となりつつある。特にスマートフォンのような新たな ICT (Information Communication Technology) デバイスによるデータの流通量は爆発的に拡大している。加えて、オープンソースソフトウェアやクラウドコンピューティングの普及、人工知能等に関する技術革新も起きたことで、大規模なデータを分析・活用してビジネスに生かす事例が急速に増えている。

データ科学とは、データの分析結果から新しい価値を生んで課題解決につなげるものである（以下、データ科学とデータサイエンスを同じ意味の用語として扱う）。データ分析を行う職種をデータサイエンティストと呼ぶことがある。データサイエンティストは高度な数学的素養、プログラミング能力、データ分析の対象となる業界知識・業務プロセス（実務者と対等に話ができる水準の専門知識）が求められる [1]。すべての能力に長けているデータサイエンティストは世界的に見ても稀であることから、一般的には複数の専門的な知識や技術を持つ人材によるチームを編成してデータ分析・課題解決に取り組む。

データサイエンスにおいて重要な役割を果たす技術の一つが、データの並列・分散処理である。本実験ではデータサイエンスの全体像を理解した上で、並列演算処理の基礎について学び、並列・分散処理の基礎の学習、および、大規模データの並列・分散処理の学習へと発展させる。

2 データ処理と分析

2.1 ビッグデータと情報抽出

ビッグデータという言葉は 1990 年代から使われ始めており、時間とともにその言葉の意味合いは変化している。総務省の情報通信白書（平成 24 年版）[3] では、文献 [4] を参照する形でビッグデータを「典型的なデータベースソフトウェアが把握し、蓄積し、運用し、分析できる能力を超えたサイズのデータ」と定義している。ビッグデータの例として、多数のセンサーから得られる時系列のストリーミングデータや、画像のデータセット、つぶやきなどのソーシャルデータなどがあり、また、各種センサーや GPS などの衛星機器、防犯カメラや医療デバイス、さらには金融市場などから送られる多様な非構造化データもビッグデータになり得る [5]。ビッグデータにはあらゆる情報を取り入れようとする大規模性があり、様々な形式、構造、計測頻度、精度などを伴った多様で不均質なものとなることが多く、データ分析を可能とするための前処理が重要な作業となる。実際にはその前処理でデータ分析に関する業務のうち八割の時間が割かれるとの報告もある。

データ分析において基盤となるデータ解析手法は主に統計的手法であり多岐に渡る。例えばデータの統計的な特徴を評価する際には基本統計量（平均、分散、標準偏差、相関係数など）を計算し、また、標本データから母集団の統計的な特徴を推測するためには推測統計学に関する知識（確率変数・確率分布、点推定、区間推定、検定など）が必要となる。データから何らかの関係を導き将来の予測をする際には一般に機械学習が利用される。機械学習は教師ありと教師なしの手法に分かれる。教師ありの手法ではクラス判定を行う分類（e.g., ロジスティック回帰）、数値を予測する回帰分析（e.g., 線形回帰、決定木）がある。教師なしの手法は主にグループ化を行うクラスタリング手法（e.g., k-means 法）がある。近年の人工知能ブームを加速させた深層学習は多層ニューラルネットワークにもとづく機械学習手法（教師あり、特徴量を自動抽出）である。

ビッグデータから何か役に立つ情報が得られないかという発想でデータ分析を始めるとうまく進まないことがあるため注意が必要である。そこで、データ分析によって解決したい（ビジネス）課題を常に意識しておき目標設定をしておくことが重要である。データサイエンスとは、適切なデータ・分析手法を選択して

データ（入力と出力）における因果関係を導き、不確かさの程度を踏まえながら入力から出力の予測、何らかの意思決定、課題解決をするための情報を提供するものである。ただしデータ選択・分析・予測・評価のプロセスは一直線に進むわけではなく何度も繰り返し試行錯誤を行い最適なプロセスにたどり着く [6]。

2.2 データ分析処理基盤

データサイエンスではデータ分析のための処理基盤が重要な役割を果たし、ハードウェアやネットワークの技術進歩とともに過去から現在まで大きく変遷している。現在でも標準的なものがあるとは考え難く、今後も各種技術の進歩とともに変化していくと予想される。以下、データ分析基盤に関する過去から現在までの変遷について簡単に解説する。以下内容の詳細は [7] を参照。

まず 1980 年代後半から多変量解析をはじめとする統計分析処理が PC 上で行われるようになり、その後 1990 年代の中盤から、RDBMS（リレーショナルデータベース管理システム）をベースとした分析専用 DB である DWH（データウェアハウス）の導入が進んだ。

2000 年代に入ると分析のためのハードウェアはデスクトップからサーバへ移行し、データマイニングや機械学習といった高度な情報分析技術がビジネスでも利用され始める。この頃から Hadoop の MapReduce 機構や NoSQL(Not only SQL) データベースが普及し始めた。MapReduce 機構は複数の汎用サーバでデータの検索・集計を分散処理する技術（詳細は後述）、NoSQL データベースは RDBMS のような SQL データベース以外のデータベース管理システムの総称であり、一般に並列分散処理機構に加えテキスト・画像・音声といった非構造化データを扱える。さらには時系列に発生するストリームデータをリアルタイムに処理する技術で CEP（複合イベント処理）も実用化され始める。

2010 年代に入り Hadoop の MapReduce やコアプロセッサを活用して大量データの計算処理を高速化する HPC（ハイパフォーマンスコンピューティング）の技術もビジネス分野に普及した。深層学習の活用の試みも進み、米 IBM の Watson や米 Google による画像の自動認識等への応用が代表的である。また分析処理（Analytics）をデータベースの中（In-Database）で実行する技術である In-Database Analytics も注目されている。データがすでに蓄積されている DWH（データウェアハウス）内で分析を行うことでデータの入出力時間が節約でき大量データを用いた処理が容易になる。

現在のビッグデータ時代において、MapReduce にもとづく Hadoop はバッチデータの並列・分散処理でよく利用されており、構造化データを対象とすることが多い。ただし、MapReduce ではメモリの制限を回避するため中間ファイルをディスクに書き込む動作が必要となる。一方、近年では HPC に搭載できるメモリのサイズも大きくなってきたことから、ディスクへのアクセスが不要な並列・分散処理機構である Apache Spark も利用されるようになりつつある。これら並列・分散処理に係る分析基盤の移り変わりは激しく、ハードウェアの発展とともに今後も変化し続けられると思われる。

2.3 データの可視化、分析結果の可視化

データの分析結果を理解するうえで重要なことがデータの可視化である。膨大な高次元データや計算結果を人間が把握できるようにするための重要な技術であり、ヒストグラム、散布図、等高線図、ヒートマップ、箱ひげ図等が代表的なものである。

2.4 データサイエンティストに必要な知識・技術

データサイエンティストに必要とされる代表的な知識・技術を以下に挙げる。ここでのデータサイエンティストはビジネス課題を解決することを目的としたものである。

1. 対象分野の専門知識
2. データ分析手法
3. ビッグデータ処理手法
4. システム構築技術

基本的にはすべてデータサイエンティストとして仕事をこなしながら習得すべき能力である。データ分析手法には統計学、機械学習、可視化に関する知識が重要となる。ビッグデータ処理手法には、ペタバイト級の散在するデータを処理するために必要な並列・分散処理、ストリーミング計算、巨大データベース処理、クラウド計算などが含まれる。本講義ではビッグデータ処理手法に焦点を当て、特に並列・分散処理を中心に学ぶ。加えてデータサイエンティストには一般的な素養・能力として以下が重要とされている。

- 情報セキュリティの知識
- 研究倫理
- 戦略立案能力、問題発掘・企画能力、問題解決能力
- データ収集能力、キュレーション能力（データの選択、前処理等）
- 異分野研究者・事業者との連携能力

3 並列演算処理

膨大なデータの処理やシミュレーションにともなう複雑な計算を行う場合、処理や計算を実行するために必要な時間をいかに短縮するかが課題となることがある [8]。処理や計算の時間を短縮するための方法はハードウェアのアップグレード等様々存在するが、まずは与えられた計算機環境で高速化することを考えるのが一般的な方法であり、その 1 つの方法が計算コードの並列化である [9–12]。また、数十年前から CPU のマルチコア化が進み、並列計算による効率化も様々な計算機プラットフォーム上で可能となり、ハードウェアの性能を十分引き出すためにも並列化の必要性はますます高まっている [13, 14]。

そこで、本章ではマルチコア環境での並列化を体験しながら学習する。開発環境として Matlab Parallel Computing Tool Box (PCTB) を使用し、簡単な例を用いて Matlab 上での並列化手順や並列化の効果・効率 [16] について理解を深める。

3.1 並列演算処理とは

並列処理とは複数のプロセッサコアを協調動作させてひとつのタスクを実行することである (図 1)。例えばあるプログラム内で 10000 個の浮動小数点の総和を計算するというタスクを考える。もし 2 個のコアを用いて 1~5000 番目までの総和と 5001~10000 番目までの総和を別個に計算して最後にこれらを合算して答えをする場合、理想的には処理速度は 2 倍となる (処理時間は半分)。しかし 2 個のコアがメモリやキャッシュなどのリソースを共有している場合や、最後の足し合わせにかかる時間コストまで考えると現実的には処理速度は倍には満たない。さらにコアの数を増やしていくと各プロセッサでの総和計算に要する時間コストはほとんどゼロにできるものの足し合わせや、その他の並列化されていない処理のコストは残り続けるため速度向上率は鈍化する。

処理の内容によってはアルゴリズムの性質上、並列処理が不可能な場合もある。例えばフィボナッチ数列を計算するというタスクを考える。N 番目のフィボナッチ数は N-1 番目と N-2 番目のフィボナッチ数の和

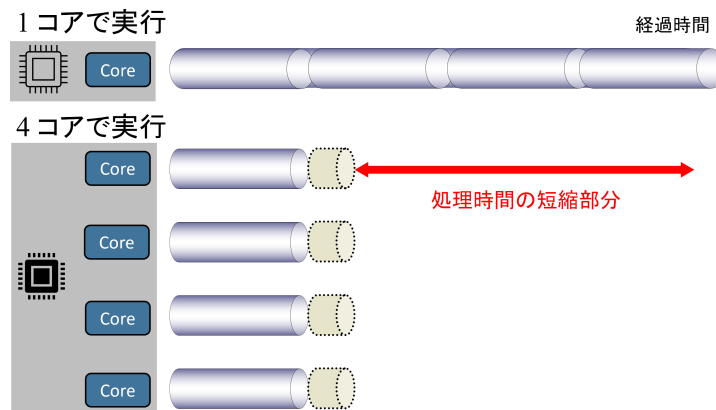


図 1: 並列演算処理の概念図

であるため事前にこれらを計算しておくことが必要である。10000 番目のフィボナッチ数を求めるとき、2つのプロセッサを使って 1～5000 番目までのフィボナッチ数と 5001～10000 までのフィボナッチ数を同時に別個に計算することはできない。5001 番目のフィボナッチ数の計算には 5000 番目と 4999 番目のフィボナッチ数が必要だからである。こうした順序依存性のある計算は並列化が困難であることが多い。またファイルの入出力のような処理も性質上並列化が困難である。このようにプログラム内には並列化できない／されていない処理部が少なからず存在するため、N 個のプロセッサコアを用いても N 倍未満の処理速度向上しか得られない。

3.2 並列計算機の分類

並列計算機とは複数のプロセッサコアを協調動作させてひとつのタスクを実行するような計算機の総称である。こうした並列計算機は大別して共有メモリ型と分散メモリ型に分類される。また、CPU（中央演算装置：いわゆる一般的なプロセッサ）とは別に多数のプロセッサからなる外部演算装置（アクセラレータ）を備えたアクセラレータ型も広く利用されている。

共有メモリ型は 1 台のコンピュータの中に複数のプロセッサを内蔵する形態を取り、一方分散メモリ型は複数の単体コンピュータをネットワークで繋ぎ合わせたもので一般にクラスタと呼ばれる。また、CPU に加えて GPU（Graphics Processing Unit）や Cell Broadband Engine など CPU 外部の高速処理装置を活用するアクセラレータ型は特性の異なる複数のプロセッサを協調動作させることからヘテロジニアスコンピュータとも呼ばれる。

3.3 共有メモリ型並列計算機（SMP）

共有メモリ型並列計算機は複数の CPU と大規模なメモリで単一のシステムを構成する方式の並列計算機である（図 2）。この方式では全 CPU が広大なメモリ空間を共有しており、各 CPU は共有メモリ空間のどのアドレスにも直接的にアクセスしデータを読み書きすることができる。共有メモリ型並列計算機の特徴は、これら複数の CPU と大容量のメモリが単体の OS によって制御されている点である。計算機のうち、どの CPU がどのメモリアドレスにアクセスする場合にも均一で対称的にアクセスが可能なアーキテクチャのことを対称型マルチプロセッシング（Symmetric Multi Processing）という。ただし、現在のほとんどの共有メモリ型計算機は物理的には CPU とメモリが密に結合された対（これをノードと呼ぶ）を複数連結す

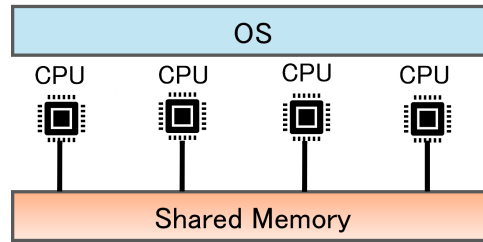


図 2: 共有メモリ型並列計算機概念図

ることで構成されており、このとき CPU が自ノード内のメモリにアクセスする場合に比べて他ノードに属するメモリにアクセスする場合は帯域（bandwidth）と遅延（latency）の両面で低速となる．このように各 CPU がどのメモリアドレスにアクセスするかによってアクセスコストが均一ではないシステムアーキテクチャのことを NUMA（Non-Uniform Memory Access）という．

共有メモリ型並列計算機では大規模なメモリ空間に対し透過的にアクセスすることが可能であるためユーザーは通常のパーソナルコンピュータの延長のようにマシンを利用することが可能であり，なおかつパーソナルコンピュータでは処理不可能な大規模な計算であっても大容量メモリを用いることによって処理することが可能である．また，全メモリ空間が連続的・透過的であることから OpenMP などのスレッドプログラミングによって容易に大規模並列計算を実現可能であるという利点をもつ．

3.4 分散メモリ型並列計算機（DMP）

分散メモリ型並列計算機は単体計算機として独立した複数の計算ノードをネットワークを介して連結し相互に通信可能としたものである（図 3）．個々の計算ノードはそれ自体独立した単体計算機としての機能を備えており，各計算ノード内の CPU やメモリといったリソースはノードごとに存在する OS によって個々に管理されている．クラスタ型と呼ばれることもあり一般的なパーソナルコンピュータをネットワークで相互接続した PC クラスタも分散メモリ型並列計算機に分類される．近年ではマルチコア CPU・マルチソケットサーバが普及したことから，各計算ノード内は NUMA 型の共有メモリアーキテクチャを取ることが多い．各計算ノード間の通信を実現するネットワーク回線のことをインターコネクトと呼ぶ．代表的なインターコネクトとして Ehternet・Myrinet・Infini Band などがある．近年では高速・低遅延なインターコネクトとして Infini Band の利用が標準的である．分散メモリ型並列計算機では，あるノードの CPU は他ノードのメモリ上のデータに直接アクセスすることはできず，インターコネクトを介して他ノード上のデータを自ノードのメモリへ転送し，結果的には自ノード内メモリのデータとして利用することになる．このデータ通信は MPI（Message Passing Interface）などを利用したプログラムにより制御可能であるが共有メモリ型のスレッドプログラミングに比べて高度なプログラミング技術が必要となる．

3.5 アクセラレータ型計算機

先述した共有メモリ型・分散メモリ型並列計算機はどちらも CPU によってメインメモリ（CPU からアクセス可能なメモリ）のデータを処理するものであった．一方，CPU 以外的高速演算機を計算ノード内部（または外部）に搭載しこの演算機（図 4）を活用することで純粋な処理性能あるいはコストあたりの性能，電力あたりの性能を高める方法も広まりを見せている．この外部の高性能演算器をアクセラレータと呼ぶ．特に，畳み込み演算が必要な深層学習ではその威力を発揮する．

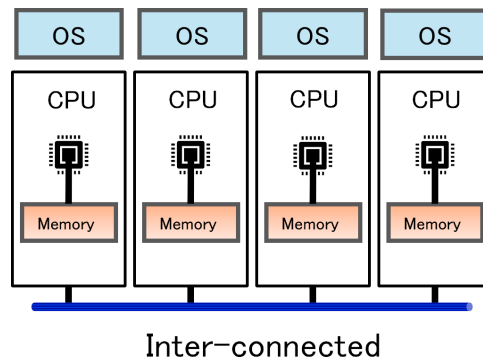


図 3: 分散メモリ型並列計算機概念図

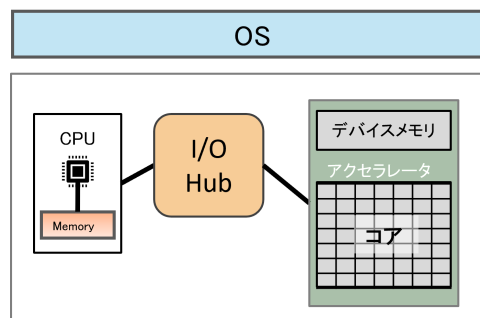


図 4: アクセラレータ型計算機 (GP-GPU) の概念図

一般的なアクセラレータとして GPU (Graphics Processing Unit)・Cell BE・FPGA などがある。アクセラレータ型計算機ではこれらアクセラレータのもつ数百に及ぶ多数のコア、消費電力の少なさ、高速なメモリアクセス性能など、CPU とは異なるデバイス特性を活用することにより高速な処理を実現する。ただし、1 計算ノードあたりに搭載可能なアクセラレータの個数には限りがあるため、アクセラレータ型計算機をインターコネクト接続しクラス構成とすることで大規模アクセラレータ型計算機を構成している。

4 情報科学部での Matlab 並列計算

4.1 演習室 3 (演習室 5,6) WS 環境での Matlab 並列計算

演習室 3 の WS 環境では 16 並列による並列演算まで可能であり、32GB のメモリが搭載されている。演習室 5 または 6 では 4 並列による並列演算が可能である。Matlab による並列計算は Matlab Tool Box の 1 つである Parallel Computing Tool Box (PCTB) を利用して行う。以下に、演習室 3 での利用可能な最大コア数と PCTB を用いた並列計算の実行手順を下記に示す。

1. 並列計算実施のための環境設定： `parpool(16)`
2. プログラムの実行
3. 並列計算実施環境の終了： `delete(gcp)`

ただし、演習室 5 または 6 の場合は、`parpool(4)` で 4 並列による並列演算処理が実施できる。上記の 1 を開始後、Linux のターミナル から `ps` コマンドを実行し、Matlab に関連するプロセス数を確認すると 16 つの Matlab プロセスが生成されているのが確認できる。また、並列計算実施環境を終了した後、再度ターミナルからプロセスを確認すると、Matlab のプロセスが 1 つになっていることが確認できる。

4.2 並列計算機

情報科学部には並列演算高速サーバが設置されている。現在の並列計算機は分散メモリ型とアクセラレータ型の機能が共存したクラスターであり、CPU による並列演算が可能なノード（CPU 計算ノード）が 20 台と、GPU (NVIDIA Tesla V100) を 2 機搭載したノード 4 台で構成されている。当 GPU は理化学研究所が運用するスーパーコンピュータの富岳でも採用されている高性能なものである。これらのシステムにより、CPU での並列演算は最大 800 コアを利用でき、複数の GPU を用いた並列演算処理も可能である。ただし、原則として研究室配属以降、指導教官を通じてアカウントの発行や利用が可能になる。

5 並列化計測・高速化予測のための指標等

1 つのコードは並列化によってどの程度まで高速化できるのだろうか。その間に答えるには並列計算を単純にモデル化し、並列化に関するさまざまな指標等を理解する必要がある。ここでは速度向上度、線形性、並列化効率などの用語について説明する。また、それらを用いて与えられえたコードの並列化結果を議論するために必要な実行効率等について解説する。

5.1 速度向上度 S_N ，理論上達成可能な速度向上度 S_N

あるプログラムを 2 並列（2 つの CPU または Core）で実行する場合を考えよう。この場合、2 並列だからと言って必ずしも 2 倍の高速化を達成できるとは限らない。それどころか、多くのケースでは理論上達成可能なレベルまで高速化を進めるのは難しいのが一般的である。このことを理解するために、まずはじめに、理論上達成可能な高速化とはどのように求めることができるのか、例を用いて説明する [13, 14, 16]。

並列化可能な処理ブロック (ParProc) (for-loop 等を含む) の前後に並列化ができない処理ブロック (PreProc, PosProc) が配置されているプログラムがあるとする。逐次処理の場合にそれぞれのブロックの実行に要する時間を T_{par} , T_{pre} , および T_{pos} と定義すると、 N 並列によって理論上達成可能な処理時間は次式となる。

$$T_N = T_{pre} + T_{pos} + T_{par}/N \quad (1)$$

ただし、 T_N には並列計算のための準備や通信に要するオーバーヘッド等は含まれていないため、実際の実行時間 t_N とは等しくならないことに注意する。

したがって、並列化による達成可能な速度向上度として定義される速度向上度 (S_N) は次式で表される [16]。

$$S_N = \frac{T_{pre} + T_{pos} + T_{par}}{T_{pre} + T_{pos} + T_{par}/N} \quad (2)$$

$$= \frac{1}{r + (1-r)/N} \quad (3)$$

ただし

$$r = \frac{T_{pre} + T_{pos}}{T_{pre} + T_{pos} + T_{par}} \quad (4)$$

式 (3) はアムダールの法則 (Amdahl's law) [14,16] として知られており、また、式 (4) の r はアムダール率 (Amdahl's fraction) [16] と呼ばれている。アムダール則から、並列化ができない処理に要する総計算時間 ($T_{pre} + T_{pos}$) が並列化可能な部分に要する時間 (T_{par}) に比べ著しく小さい場合 ($T_{pre} + T_{pos} \ll T_{par}$) S_N は N に近づくことになる。逆に、 r が比較的大きい場合や r が 1 に近い場合 ($T_{pre} + T_{pos} \gg T_{par}$) には並列数 N を高めたとしても、達成可能な高速化は限定的となる。一般的には、 N 並列によるコードの実行時間 (t で表される時間は、理論的な実行時間ではなく、実際の計算機プラットフォーム上で実行された際に計測された時間) を t_N とすると、通信による遅延等の影響があるため (次節で詳細を説明)、 $s_N = t_1/t_N$ は S_N より小さくなる。この仮想的な例を通した考察から、並列化の方針を決定する際には次の 2 点が重要であることが理解できるであろう。

- 最も時間を要する処理ブロックを認識し並列化を検討する (並列化の可否も含めて検討する)。
- 達成可能な速度向上度 (S_N) を概算することでそれぞれのケースにおける高速化の限界を認識し、その値に近づけることを目標にする。(コア数が N 個であっても N 倍の高速化が実現できるわけではない！)

5.2 線形性、並列化効率 (E_N)

並列化により達成された実際の速度向上度 (高速化率) s_N が、並列化により理論上達成可能な高速化 S_N に対してどの程度であるかを示す指標として並列化効率がある。並列化効率は

$$E_N = s_N/S_N, \quad (E_N \leq 1) \quad (5)$$

で定義され、その値が 1 に近いほど効率良く並列化できていることになる。

ところが、実際には前述したように s_N と S_N の間には差があり、その差は通常 N とともに大きくなる。並列数 N が小さい場合には N 増加とともに s_N も直線状に増加するが、そのことを並列計算における線形性と言う [13,14,16]。実際には並列数が大きくなると、それ以上並列数を増やしても十分な効果が得られない。つまり、プログラムによっては並列数を増やしたとしても理論上達成可能な高速化 (S_N) から一層離れることになり、期待したほどの“投資効果”が得られないことになる。線形性を有する場合 (もしくはその範囲内の並列数である場合) には計算ノードを増やすなどの“投資効果”が得られることになるので、並列化における線形性の調査はその意味で重要となる。

5.3 実行効率 (e_N)、スケーラビリティ (scalability)

一方、あるプログラムを並列数 N で並列化することを考えた場合、費用対投資効果の観点からは速度向上度が N 倍になることを期待するので、速度向上度が N 倍にどの程度近いのかということも重要な指標である。つまり、 s_N/N を予測しておく必要がある。 s_N/N は実行効率とよばれる指標で [14]、ここでは e_N で定義する。

$$e_N = s_N/N \quad (6)$$

実行効率が 1 に近い場合、与えられたコードは並列数を増加させたとしても費用対効果を十分見込める可能性が高い。実行効率が 1 に近いことを並列数に対してスケーラブルであると言う [13,16]。ただし、ここでは繰り返し計算などの処理速度がスケーラブルなだけで、並列数を増やす場合には取り扱うデータの量も同時に増えていくことを考慮する必要があるのでスケーラビリティを議論する際にはその点に関する注意を要する (Gustafson-Barsis's law) [13]。

5.4 アムダール率 (r) の近似と並列化効率の予測について

線形性 (linearity) については s_N をプロットするだけで調査することが可能であるが、並列化効率に関しては S_N を概算する必要がある。逐次計算により S_N を概算する方法は、繰り返し計算の回数を変化させる方法等が考えられるが、適当な並列数 n の場合の計算結果 t_n と逐次計算による結果 t_1 からアムダール率 r を次のように近似的に推定することができる。アムダール則 (式 (3)) を r について解くと次式を得る。

$$r = \frac{N/S_N - 1}{N - 1} \quad (7)$$

式 (8) 中の S_N を $s_n = t_1/t_n$ で置き換え、 r を近似する。

$$r \approx \frac{n/s_n - 1}{n - 1} \quad (8)$$

つまり、比較的小さいある並列数 n の結果と逐次計算の結果から式 (8) で r を近似し、それを用いて S_N を概算することができれば、実際の速度向上度 s_N と比較することで並列化効率、および、実行効率を概算することができる。ただし、このようにして求める r は並列オーバーヘッドの影響が小さいときに有効であり、データ処理を伴う並列処理を考える際は、並列オーバーヘッドの存在を考慮して考える必要がある。

その他、逐次計算を 1 回行い、その結果からアムダール率 r を直接計測する方法もある。

5.5 実験用プログラム

5.5.1 練習用プログラム 1

```
% demo1.m

for n = 1:4
    pause(rand)
    n
end
```

5.5.2 練習用プログラム 2

```
% demo2.m

f = zeros(1,15);
f(1) = 1;
f(2) = 2;

for n = 3:15
    f(n) = f(n-1) + f(n-2);
end

f
```

5.5.3 練習用プログラム3

```
% demo3.m

A=zeros(1,4);
d = 0; n = 0;

for n = 1:4
    d = n*2;
    A(n) = d;
end

A
d
n
```

5.5.4 program1.m

```
% program1.m

n1end=120;

t0=0.5; t1=0.02; %case1:
%t0=0.02; t1=0.5; %case2:

tic

pause(t0)

for n1=1:n1end %parfor ... (Line-A)
    pause(t1)
end

toc
```


5.5.5 program2.m

```
% program2.m

n1end=200; n2end=16; %case1:
%n1end=20; n2end=160; %case2:

t1=0.000001; t2=0.01;

tmp=zeros(1,n1end);
tic

pause(1)

for n1=2:n1end %for ... (Line-A)
    pause(t1); tmp(n1)=2.0*tmp(n1-1);

    for n2=1:n2end %for ... (Line-B)
        pause(t2)
    end
end

end

toc
```

5.5.6 program3.m

```
% program3.m

n1end=4; n2end=400; %case1:
%n1end=400; n2end=4; %case2:

t1=0.000001; t2=0.01;

tic

pause(1)

for n1=1:n1end %for ... (Line-A)
    pause(t1)

    for n2=1:n2end %for ... (Line-B)
        pause(t2)
    end
end

end

toc
```

5.5.7 モンテカルロ法による円周率の計算プログラム

下記プログラムを実行する前に `format long` を実行すること。

```
% Monte Carlo code for PI computation

nhist=1e5;

count=0;

tic;

for i=1:nhist
    length=norm(rand(1,2)); % ||r||

    if (length<=1)
        count=count+1;
    end
end

toc

res=count/nhist*4

res-pi
```

5.6 実験課題

【課題 1】 r と N を変数として S_N をプロットせよ。

【課題 2.1】 指導書中の `program1.m` には並列化可能な `for-loop` が含まれている。下記の手順にしたがって、このプログラムの並列化についての調査を行い、任意の並列数 N の時の並列化結果について検討せよ。

(手順 1) `for-loop` 中の `index(n1end)` を適当な値に書き換え、`program1.m` の逐次計算を実行し、各 `n1end` の時の実行時間を表にまとめよ。

(手順 2) 手順 1 で作成した表からアムダール率 r を求めよ。

(手順 3) 手順 2 の結果にもとづき、任意の N に対する高速化指標 S_N を概算し表にまとめよ。

(手順 4) プロファイラを用いて手順 1 の結果を検証せよ。(結果の妥当性についての検証。)

(注) レポートには結果だけでなく手順および概算に使用したデータ等を示し、概算結果の根拠を示すこと。

【課題 2.2】 `matlab` のコマンドウィンドウで `matlabpool open 1` を実行し、`matlab` の並列プロセス (worker) を 1 つ生成した状態で上記の課題 2.1 の手順 1 から 3 を再度実行し、その結果を課題 2.1 の結果と比較・検討せよ。なお、worker 数は、`linux` の terminal からコマンド `ps` を実行することで確認せよ。また、計算実行の際には、`for-loop` の `'for'` を `'parfor'` に書き換えよ。

【課題 3】 指導書中のプログラム (parprog2.m) には並列化可能な for-loop が 1 つ含まれている。このプログラムを並列化した場合の理論上達成可能な速度向上度を概算せよ。(まず、並列化可能な for-loop を探し、課題 2 と同様に for-loop に使われているインデックスを変化させながら逐次計算を実行し、アムダール率 r を算出せよ。また、並列数を増加させたときの理論上達成可能な速度向上度を予測せよ。最後に、プロファイラを用いて結果の妥当性を検証せよ。

【課題 4】 3つのプログラム (program1.m, program2.m, program3.m) を parfor により並列化し、速度向上度、実行効率、並列化効率について調査し、線形性、および、スケーラビリティについて検討せよ。(並列数を増やした場合の結果を予測せよ。) 検証が必要な場合には、分散メモリ型並列計算機を利用しても良い。ただし、プログラム中の $t1$, $t2$, $n1end$, $n2end$ を変化させ、これらプログラムの並列化について調査せよ。

【課題 5】 モンテカルロ法による π の計算プログラムを並列化し、このプログラムの並列化について調査・検討せよ。特に、速度向上度、実行効率、並列化効率について調査せよ。

【課題 6】 上記の【課題 5】を並列計算サーバを用い $N = 64 \sim 256$ の並列数で実行することを考える。試行回数 (nhist) を 10^{10} とした場合の計算時間、および、速度向上度を予測せよ。(予測した根拠を示すこと。)

【課題 7】 マンデルブロ集合を計算し、表示するプログラムを作成し、それを並列化せよ。次に、このプログラムの並列化について調査・検討せよ。また、どのようなサイズの縮尺であってもフラクタルが続く様子を動画にしたい。虚数空間上の 1 点を固定し、その点を中心に整数部の範囲と虚数部の範囲を変更することで表示する部分の縮尺を変化 (拡大) させ、その結果を動画にせよ。最後に、並列計算により実行時間がどの程度短縮できるのか報告 (または予測) せよ。

【課題 8】 モンテカルロ法による輸送計算コードの並列化を試み、このプログラムの並列化について調査・検討せよ。

【課題 9】 大気中の微粒子が移動・拡散する様子をシミュレーションするためのプログラム (移流拡散方程式を有限差分法で解くための計算コード) の並列化について調査・検討せよ。

6 並列オーバーヘッド

6.1 並列オーバーヘッドの要因

処理の並列化を行うとき、速度向上度が並列数に一致するようなことは一般的にあまりなく、実際には速度向上度は並列数より小さくなる。並列化により生じる処理時間の遅延を並列オーバーヘッドと呼ぶ。並列オーバーヘッドには以下に示す様々な要因があると考えられる。

- プロセス間での通信による遅延
- データ分散時のメモリ確保
- 同期 (複数スレッドが強調する動作のことで、共有リソースへの排他制御など)
- 並列処理プログラムの処理時間の不均一性

並列処理時にはワーカー間での通信やクライアント・ワーカー間での通信が発生する。これはハードウェアの性能にも大きく依存する。CPU による演算ではレジスタ、キャッシュ、メインメモリ等に格納された情報やデータを利用する。レジスタへのアクセス速度はメインメモリより早い一方、容量はメインメモリの方が大きい。したがって、各記憶領域へのアクセス速度等のハードウェアの性能が低いと、並列処理時に通信が増えれば増えるほど並列オーバーヘッドは大きくなると考えられる。比較的高性能なシステムを用いてもこれらの影響は多くの場合発生する。

また、大きなデータのやり取りがクライアント・ワーカー間で発生する場合（データ分散時など）にはその影響は大きい。特に、データを分散するためにメモリ確保を都度行う必要があることがあるため、効率的なメモリの管理が必要である。

実際の処理で並列オーバーヘッドの影響を考える場合、各々の要因が与える遅延の度合いを定量的に把握することは難しいが、処理プログラムの性質から各影響の度合いを予測することは可能である。

6.2 Karp-Flatt metric

ここでは並列オーバーヘッドの存在を想定した並列化評価指標 Karp-Flatt metric について述べる。まず先に、Karp-Flatt metric (K_f) が次式で計算できるものとする [17]。

$$K_f = \frac{\frac{1}{s_n} - \frac{1}{n}}{1 - \frac{1}{n}} \quad (9)$$

s_n は並列数 n での速度向上度である。

$$s_n = \frac{T_s}{T_n} \quad (10)$$

T_s は逐次処理時間、 T_n は n 並列での並列処理時間である。

上記とは別に、 K_f を次式で定義できるものとする [18]。

$$K_f = \frac{T_{prepos} + \frac{n}{n-1}T_{oh}}{T_s} \quad (11)$$

T_{prepos} は元来並列化できない部分の処理時間、 T_{oh} は並列オーバーヘッドとする。 T_{oh} や r は実際には並列数 n に依存するものである。逐次処理時間 T_s と並列処理時間 T_n は次式で定義される。

$$T_s = T_{prepos} + T_{par} \quad (12)$$

$$T_n = T_{prepos} + T_{par}/n + T_{oh} \quad (13)$$

T_{par} は並列化可能な部分の処理時間である。式 (11) より、

$$K_f T_s = T_{prepos} + \frac{n}{n-1}T_{oh} \quad (14)$$

と書ける。これを並列化できない部分の処理時間と考えることにする。並列化可能な部分の処理時間を $(1-r)T_s/n$ とすると、

$$\begin{aligned} \frac{(1-K_f)T_s}{n} &= \frac{1}{n}(T_s - K_f T_s) \\ &= \frac{1}{n} \left(T_{prepos} + T_{par} - T_{prepos} - \frac{n}{n-1}T_{oh} \right) \\ &= \frac{1}{n} \left(T_{par} - \frac{n}{n-1}T_{oh} \right) \\ &= \frac{T_{par}}{n} - \frac{T_{oh}}{n-1} \end{aligned} \quad (15)$$

と書ける．これらより，並列化できない部分の処理時間と並列化可能な部分の処理時間の和は次式のように T_n となる．

$$\begin{aligned} K_f T_s + \frac{(1 - K_f) T_s}{n} &= T_{prepos} + \frac{n}{n-1} T_{oh} + \frac{T_{par}}{n} - \frac{T_{oh}}{n-1} \\ &= T_{prepos} + T_{par}/n + T_{oh} \\ &= T_n \end{aligned} \quad (16)$$

ここで，式 (10) と式 (16) より，

$$s_n = \frac{T_s}{K_f T_s + \frac{(1 - K_f) T_s}{n}} \quad (17)$$

と書け，これを K_f について解くと次式を導ける．すなわち式 (9) に一致する．

$$K_f = \frac{\frac{1}{s_n} - \frac{1}{n}}{1 - \frac{1}{n}} \quad (18)$$

したがって，式 (11) で定義した K_f は， T_{prepos} や T_{oh} が事前にわかっていなくとも，式 (9) から経験的に計算することができる．もしくは，逐次処理時間に対する T_{prepos} と T_{oh} の割合を速度向上度から近似できることになる．

ただし，式 (11) では T_{oh} に $n/(n-1)$ がかかっているため，並列オーバーヘッドをわずかに過大評価していると考えられるが， n が大きい場合はその影響が小さい．

7 データの並列・分散処理

データサイエンスの分野では，比較的大きなデータ（例えば画像データ数 GB）を読み込んで各要素（各画素）に対して統一的な処理をしたい場合，データを分散させて並列処理することで高速化が可能となるケースがある．

しかし，データを分散させて並列処理を行おうとする際，例えば Matlab の `parfor` では使用メモリ量が使用可能な範囲を超えて処理不可能となることがある．この問題は `parfor` においてある条件下で処理を行う際，分散対象のデータ全体を各ワーカーにコピーするために発生する．

今回の実験目的は，ビッグデータの並列・分散処理を考え，Matlab で良く利用される `parfor` の限界を確認するとともにその問題に対する解決策の一つとなる関数 `spmd` について学ぶ．

7.1 準備

メモリの使用状況を確認するためのシェルスクリプト (`mem.sh`) を準備しておく．

```
for i in `seq 10800`; do
    date
    free -m
    sleep 1s
done
```

コマンドライン上で `sh mem.sh` 実行することで 1 秒ごとに日付・時刻およびメモリの使用状況が表示される．サンプリング間隔を変えたい場合は `sleep` のオプション `1s` を変更する．この場合三時間計測したらこのプロセスは終了する．

7.2 並列・分散処理時におけるデータのやりとり

並列プールを立ち上げると、Matlab のクライアントプロセスとは別に、ワーカーごとに（並列数分だけ）プロセスが立ち上がる。このワーカーが実際に並列・分散処理を行うプロセスである。そのため、クライアントで定義した、または、クライアントで読み込んだ変数・データを並列処理で利用する場合、データを各ワーカーに分散する必要がある。このとき、クライアント・ワーカー間でデータ通信のための遅延が発生する。また、並列処理終了後、処理結果をクライアントに返す際にもデータ通信のための遅延が発生する。並列処理時においてクライアント・ワーカー間のデータ通信量を如何に小さくするかは、計算機リソースを有効に使うための重要な課題である。

7.3 parfor による大規模データ処理時の問題

大規模な配列を定義し、データをブロック単位で処理することを考える。以下の例では parfor の特徴・制限が確認できる。事前に 16 ワーカーの並列プールを起動しておく。

```
% sample1.m

clear;
A=rand(16000,16000);

parfor z=1:16
    tmp=A(:,(z-1)*16000/16+1:z*16000/16)*z;
end
```

7.4 spmd とは

spmd は Single Program Multiple Data の略称で、Matlab においてはデータを分散させて並列処理を可能とするための関数である（図 5）。spmd には以下の特徴がある。

- クライアントにあるデータを各ワーカーに分散させて（分割・コピーさせて）並列処理が可能
- 各ワーカーでは独立したワークスペースを利用
- どのデータがどのワーカーに割り当てられるか認識可能
- データの分割の仕方が任意に指定可能
- ワーカー間で通信（データのやり取り）が可能
- distributed 配列を定義する際にはメモリを余分に定義（メモリ確保の時間がかかる）
- distributed 配列を分割する際には最後の次元で分割
- spmd 実行後変数はワーカーにあり、各変数にアクセス可
- gather を用いて必要なデータをクライアントへ転送

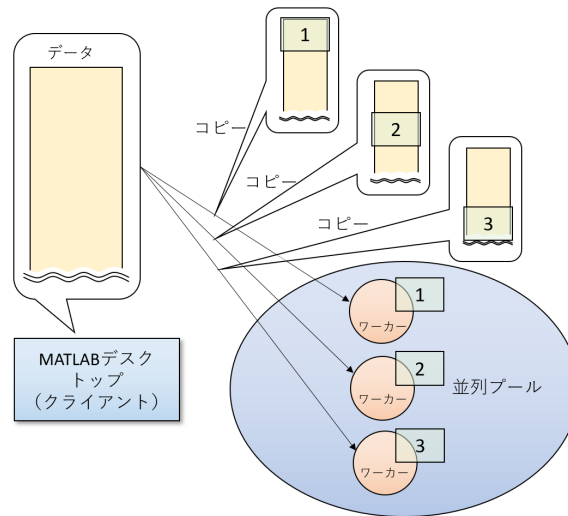


図 5: spmd によるデータ分散のイメージ

7.5 spmd の使い方

spmd は並列化対象のプログラムを spmd ステートメントで囲うことで並列化可能とする。先週の demo1.m にあるような pause 関数を含む for 文を並列化したい場合には用いない。何か処理対象となるデータがある場合に利用する。ただし、並列プールは事前に定義しておく。

```
spmd
    並列化したい処理ブロックのプログラム（データを処理するプログラム）
end
```

7.6 composite オブジェクト

spmd 実行後はデータが各ワーカーにあり、クライアントからデータにはアクセスできる。必要なデータのみクライアントに転送することになる。一方、parfor では parfor 実行中、 z 回目に処理した結果を格納した配列は実行後クライアントに返されており、parfor 中に parfor の中で定義したその他の変数にはアクセスできない。

```
% demo1

parfor z=1:16
    a=1;
    b(z)=z;
end
```

spmd を使った配列の分散を考える前に、spmd の中、つまり並列処理をする中で定義される変数について考える。この変数には spmd 終了後でもアクセスできるが、parfor ではできない。なぜ可能かという点、spmd 処理後、ワーカーが composite 型という特殊な形式で spmd の中で定義された変数を保持しているからである。並列プールが開いている限りその変数にはクライアントからアクセスできる。composite 型変数はワーカー数だけの要素数のセル配列のようなもので、その中に各ワーカーで定義された変数（配列）が格納され

ている。例えば以下の demo2.m では spmd 実施後に Matlab のコマンドラインで a{1} とタイプするとワーカー 1 に格納されている a の値が確認できる。

```
% demo2

clear;
spmd
    a=1;
    b=rand(10,10);
    c=labindex;
end
```

c は spmd ステートメントの中で各ワーカーの識別番号を返す labindex 関数の結果を格納したものである。

7.7 distributed 配列と gather

distributed 配列（分散配列）の利用により配列を複数のワーカーに分散させて並列処理が可能となる。distributed 配列は spmd ステートメントの前に定義して spmd ステートメントの中で利用する。distributed 配列が定義された時点で配列が分割され、開いている並列プールにおける複数のワーカーへその分割データが転送される。ただし、データ転送の際、各ワーカーが割り当てられたプロセスでは一定サイズのメモリ容量を確保する。課題等で distributed 配列を定義する際、分散させる元の配列が利用するメモリサイズと distributed 配列の定義により確保されたメモリサイズには一定の関係があることを確認するとよい。spmd の中では distributed 配列は codistributed 配列（対話型分散配列）として扱われる。

```
% demo3

clear;
a=ones(16000,16000);
ad=distributed(a);
spmd
    al=getLocalPart(ad);
    ad2=ad*2; % processing
    ad2_com=gather(ad2,1);
end
```

gather(ad2,1) では対話型分散配列の要素を composite 型の変数 ad2_com に格納するが、全要素をワーカー 1 に格納し、他のワーカーには空の配列を格納する。1 は labindex で識別可能な番号である。以下のコマンドによりワーカー 1 にあるデータをクライアントで取得する。

```
gad1=ad2_com{1};
```

分散配列 (ad2) を直接クライアントに転送することもできる。そのためには spmd ステートメントの後で以下の gather 関数を利用する。ただし、データのサイズが大きいと転送にはある程度の時間がかかる。

```
gad2=gather(ad2);
```

7.8 codistributed 配列

spmd の中で分散配列に相当するものを定義したい場合、対話型分散配列を作成する関数 codistributed を使う。codistributed を使うと任意の分割が可能であるため利便性が高い（distributed 配列の場合は配列

の最後の次元が自動で分割される。)

対話型分散（以下の例における `codistributor1d()`）によって定義された分散スキームを利用してワーカーに複製された配列を分散する。`codistributor1d(1)` は配列を 1 次元の方向に分割することを指す。

データ配列の位置によって処理時間に偏りがあると分割の仕方によってはワーカー間でも処理時間に偏りが生じる。そこでワーカー間で処理時間を均一になるようデータを分割すれば全体の処理時間も削減できる。

```
% demo4

clear;
a=rand(160,160);
spmd
    ad=codistributed(a,codistributor1d(1));
    al=getLocalPart(ad);
end
```

spmdに係る配列（変数）型の種類

	Composite	Distributed	Codistributed
どういう配列か？	spmdの中で発生する変数・配列を格納したもの。実態はワーカーにあり。	クライアントから見たデータ分散のための配列。定義後分散データは各ワーカーに転送。	並列プールで見たデータ分散のための配列。分散データは各ワーカーに定義。
定義する場所	spmdの前（事前に定義がなくても良い）	spmdの前	spmdの中
各ワーカーにおける配列格納方法は？	基本的に各ワーカーで同じ変数・配列を格納*。	各ワーカーで分散された（独立の）配列を格納。	各ワーカーで分散された（独立の）配列を格納。
クライアントから確認可能	○	×	×
任意の次元で分割可能	-	× 配列の最後の次元で分割	○
主な用途は？	通常の変数、分散配列の集約など。クライアントから確認。	データの分散処理。	データの分散処理。任意の方向に分割。

*ワーカーに割り当てられたインデックスなどを利用して異なる配列を格納することも可能である。

spmdの外での分散配列はDistributed、spmdの中での（対話型）分散配列はCodistributedと考える。

図 6: spmd に係る配列（変数）型の種類まとめ。

7.9 実験課題

課題実施時の注意事項として以下を挙げる.

- 各種計算は Matlab のスクリプトを作成して行うことを推奨する.
- メモリの挙動が不自然であると感じた場合は, ターミナル上で `w` とタイプし, 他にログインユーザが居ないか確認すること.
- 課題 3 の並列処理時における時間の計測は, `parpool` を起動後一度 `parfor` を実行しておき, 二度目以降の実行による実行時間を計測すること (一回目で並列処理プログラムのコンパイルが行われ, 二回目以降はその実行ファイルを利用して時間短縮されるため.)

【課題 1】

`parfor` を利用した際のメモリ使用量を示す.

$$M = m + m_c + a + n(p + a + a') \quad (19)$$

$$m_c = m' - m \quad (20)$$

M : 並列計算実行時のマシンの総メモリ使用量

m_c : Matlab クライアントのメモリ使用量

m : Matlab 起動前のメモリ使用量

m' : Matlab 起動後のメモリ使用量

a : 処理対象のデータ (配列) のサイズ

n : ワーカー数

p : 並列プールにおける一つのワーカーのメモリ使用量

a' : `parfor` 実行時余分に確保される一つのワーカー当たりのメモリ使用量

本実験においては $a' = ka$ となる k を定義する. k はワーカー数 n に依存するが, 本実験では問題を簡素化するため n に依存しないものと仮定する. また p , a' はワーカー間で異なることがあるが, ワーカー間で一定と仮定する.

以下 1~5 の時点でメモリの使用状況を記録せよ (サンプリング間隔は 1 秒程度で良い). その後, 作業 1-1 から 1-3, および, 課題 1-1 と 1-2 を行え. 4 では `program1.m` の A を定義するところまで実行したうえでメモリ使用量を確認し, 5 では `program1.m` を最後まで実行後, メモリ使用量の振る舞いを見ながら確認する (メモリ使用量が徐々に増えていくので最大値またはそれに近いと思われる値を記録する.)

1. Matlab 起動前 (計測値: m)

2. Matlab 起動後 (計測値: m')

3. ワーカー数 16 で並列プール起動後 (計測値: $m' + np$)

4. 配列 A を定義した後 (計測値: $m' + a + np$)

5. `parfor` 実行中におけるメモリ使用量の最大値 m_{max} (計測値: $m' + a + n(p + a + a')$). ただし, 徐々にメモリ使用量が大きくなることから, `free` の出力結果をよく見ながら, 最大値を記録する.

作業 1-1 1 と 2 の結果と式 (20) から m_c を求めよ.

作業 1-2 2 と 3 の結果から p を求めよ.

作業 1-3 4 と 5 の結果から a' を求めよ. a は配列 A のサイズである. また a に対する a' の比 ($k = a'/a$) を求めよ.

課題 1-1 $m, m_c, p, m_{max}, a', k$ の観測結果を表などにまとめよ.

課題 1-2 演習室 3 のデスクトップマシンにおいて 4 並列, 10 並列, 16 並列で今回の処理を行う際, parfor では本課題の前提でどの程度のサイズのデータまで仮想メモリを利用せず処理が可能か. $a' = ka$ として式 (19) から a を導出した後, 本課題で求めた m, m_c, p, k を用いて各並列数での a を計算して答えよ. ただし, M に相当する演習室 3 のデスクトップマシンのメモリ容量は 32GB である (1GB=1024MB).

```
% program1.m

clear;
npar=16;
rc=8400;
A=ones(rc,rc);

parfor i=1:npar
    tmp=A(:,(i-1)*rc/npar+1:i*rc/npar)*i;
    e(i,:)=reshape(tmp,1,[]);
end
```

【課題 2】

spmd を利用した際のメモリ使用量について考える.

$$M = m + m_c + a + n(p + a/n) + c + g \quad (21)$$

$$m_c = m' - m \quad (22)$$

$$c = a'_{sp} + b \quad (23)$$

M : 並列計算実行時のマシンの総メモリ使用量

m_c : Matlab クライアントのメモリ使用量

m : Matlab 起動前のメモリ使用量

m' : Matlab 起動後のメモリ使用量

a : 処理対象のデータ (配列) のサイズ

n : ワーカー数

p : 並列プールにおける一つのワーカーのメモリ使用量

a'_{sp} : 分散配列の定義後に分散配列のサイズ以外に確保されるメモリ領域の総量

c : spmd 実施後分散配列のサイズ以外に確保されるメモリ領域の総量

b : spmd 実施前後のメモリ使用総量の変化量

g : ワーカーからクライアントへ転送されるデータ量 (サンプリング間隔に依存)

本実験においては $g = k_{sp}a$ となる k_{sp} を定義する. k_{sp} , a'_{sp} および c はワーカー数 n に依存しないものと仮定する.

以下 1~7 の時点でメモリの使用状況を記録せよ (サンプリング間隔は 1 秒程度で良い). その後, 作業 2-1 から 2-5, 課題 2-1 から課題 2-5 を行え. 以下の 4~7 を実施するためには program2.m の必要な部分を逐次実行する.

1. Matlab 起動前 (計測値: m)
2. Matlab 起動後 (計測値: m')
3. ワーカー数 16 で並列プール起動後 (計測値: $m' + np$)
4. 配列 A を定義した後 (計測値: $m' + a + np$)
5. distributed 配列の定義後 (計測値: $m' + a + n(p + a/n) + a'_{sp}$)
6. spmd の実施直後 (計測値: $m' + a + n(p + a/n) + c$)
7. 最後の行でのデータ転送時におけるメモリ使用量の最大値 $m_{max,sp}$ (計測値: $m' + a + n(p + a/n) + c + g$). ただし, 実行時間が比較的短いことがあるため, 計測値にはばらつきがあるものと考えてよい.

作業 2-1 1 と 2 の結果と式 (22) から m_c を求めよ.

作業 2-2 2 と 3 の結果から p を求めよ.

作業 2-3 4 と 5 の結果から a'_{sp} を求めよ. a'_{sp} は a の何倍になるか求めよ.

作業 2-4 4 と 6 の結果から c を求めよ. また a に対する c の比 ($k_c = c/a$) を求めよ. また, c と a'_{sp} の差はいくつになるか求めておくこと.

作業 2-5 6 と 7 の結果から g を求めよ。また a に対する g の比 ($k_{sp} = g/a$) を求めよ。

課題 2-1 $m, m_c, p, a'_{sp}, c, k_c, m_{max,sp}, g$ の観測結果を表などにまとめよ。

課題 2-2 program2.m の 8 行目の「a=distributed(A);」から 15 行目の「e=d{1};」までの部分で具体的にどのような処理が行われているか説明せよ。

課題 2-3 演習室 3 のデスクトップマシンにおいて 4 並列, 10 並列, 16 並列で今回の処理を行う際, spmd では本課題の前提でどの程度のサイズのデータまで仮想メモリを利用せず処理が可能か. distributed 配列定義後にクライアントにある配列 A (式 (21) の右辺第三項 a) を消去できると考えて, $g = k_{sp}a$ および $c = k_c a$ として式 (21) から a を導出した後, 本課題で求めた m, m_c, p, k_c, k_{sp} を用いて各並列数での a を計算して答えよ。

```
% program2.m

clear;
npar=16;
rc=8400;
A=ones(rc,rc);

a=distributed(A);

spmd(npar)
    b=labindex;
    c=a*b;
    d=gather(c,1);
end
e=d{1};
```

8 ビッグデータの並列・分散処理

本節では、メモリに収まりきれないデータの処理を想定し、(ビッグ) データの並列・分散処理方法について学ぶ。特に、ビッグデータのバッチ処理でよく利用されている処理アルゴリズムの代表格である, MapReduce について紹介する。近年では、この MapReduce に似たアルゴリズムでさらに効率の良い処理をするミドルウェアや処理方法 (Apache Spark など) も提案されており、時代とともに急速に発展している。しかしながら、ベースとなる考え方はいずれも似通っており、データのチャンクを各コアに分散させて処理を繰り返すという点では共通していることが多い。

8.1 datastore

大規模なデータを対象とした処理を考える。datastore 関数によってすべてのデータをメモリに読み込まなくても対象のデータをプレビューできる。ここでは航空路線のデータセットを読み込む例を挙げる。ただし、この例でのファイルサイズは 11MB 程度である。この処理は次節以降の MapReduce や tall 配列の利用が必要となる。

```
ds=tabularTextDatastore('airlinesmall.csv');
```

SelectedVariableNames プロパティを使用して目的の変数 (列) を指定し、その変数だけをプレビュー表示したり読み取ることが可能である。

```
ds.SelectedVariableNames={'DepTime','DepDelay'};  
preview(ds)
```

8.2 MapReduce

MapReduce [19] によって”メモリに収まりきらない”大規模データを並列・分散処理することが可能である。spmd では並列・分散処理が可能であるが、実行時にメモリに収まるデータのみが対象となる。MapReduce ではメモリ容量に関する制限がない（どんなに大きなサイズのデータでも構わない）。

本節では航空路線のデータセットで最長のフライト時間を検索することを考える。まず map 関数により、datastore からのチャンクごとに最大値を計算する。次に reduce 関数が map 関数の呼び出しで計算されたすべての最大値の中での最大値を計算する。

まず map 関数 maxTimeMapper.m を作成する。以下の例では、'MaxelapsedTime' というキーと最大遅延時間の値を値として関連付け（マップ）を行う [20]。これは以下三つの入力引数をとる。

1. 入力データ。これは、関数 read を datastore に適用して取得されたテーブルである。
2. 構成およびコンテキスト情報のコレクション、info。今回は必要としない。
3. 中間データストレージオブジェクト。map 関数からの計算結果を記録する。関数 add を使用して、キーと値のペアをこの中間出力に追加する。

カレントディレクトリに次の map 関数 (maxTimeMapper.m) を保存する。

```
function maxTimeMapper(data, ~, intermKVStore)  
% Copyright 2014 The MathWorks, Inc.  
  
maxElapsedTime=max(data{:, :});  
add(intermKVStore, 'MaxElapsedTime', maxElapsedTime);  
end
```

次に、reduce 関数を作成する。map 関数で作成されたキーと値のペアを利用し最終的な集計を行う。三つの入力引数がある。

1. 一連の入力 ”キー”。この例のような単純な問題では無視できる。
2. mapreduce が reduce 関数に渡す中間データ入力オブジェクト。このデータはキー/値のペアの形式をもち、関数 hasnext および getnext を使用して各キーの値を反復処理する。
3. 最終的な出力データ ストレージ オブジェクト。関数 add および addmulti を使用して、キーと値のペアを出力に直接追加する。

カレントディレクトリ/フォルダに次の reduce 関数 (maxTimeReducer.m) を保存する。

```
function maxTimeReducer(~, intermValsIter, outKVStore)  
% Copyright 2014 The MathWorks, Inc.  
  
maxElapsedTime=-inf;  
while hasnext(intermValsIter)  
    maxElapsedTime=max(maxElapsedTime, getnext(intermValsIter));  
end  
add(outKVStore, 'MaxElapsedTime', maxElapsedTime);  
end
```

新たに m ファイルを作成し, datastore で csv データを読み込み, 処理対象の変数を指定する. さらには mapreduce 関数を使用して mapreduce を呼び出せるようになる.

```
% file name

ds=datastore('airlinesmall.csv', 'TreatAsMissing', 'NA');
ds.SelectedVariableNames = {'ActualElapsedTime'};

result = mapreduce(ds, @maxTimeMapper, @maxTimeReducer);
```

最後に関数 readall を追記して, mapreduce アルゴリズムの結果を表示できる.

```
readall(result)
```

図 7 は MapReduce の処理例を図で示したものである. 飲料のデータから種類ごとにその数をカウントするものである.

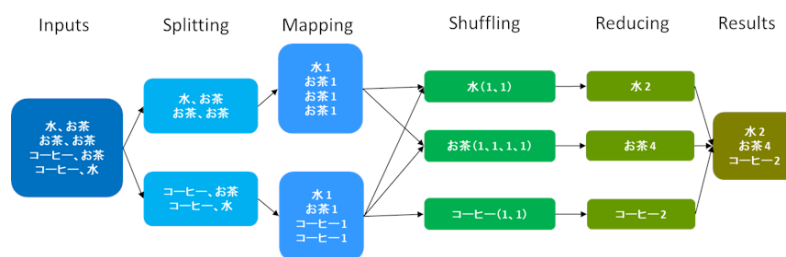


図 7: MapReduce の処理例.

8.3 クライアント・ワーカー間におけるデータ転送量の確認

ticBytes(gcp) と tocBytes(gcp) というコマンドを利用することで, 並列・分散処理時におけるクライアント・ワーカー間でのデータ転送量を計測できる. それらのコマンドを転送量を確認したい前後の行に記述しておくことで計測結果を確認できる. ただし, tocBytes(gcp) のあとにセミコロンを付けずおくことでコマンドウィンドウで計測結果が確認できる.

```
ticBytes(gcp)
計測対象の並列・分散処理スクリプト
tocBytes(gcp)
```

8.4 実験課題

米国ニューヨーク市が公開するタクシーに関する csv 形式のデータ (約 800MB) を複数利用する [23]. 今回はメモリに収まるか否かは関係なく MapReduce を利用することとする. データの項目としては, 乗車/降車した地域の種別を表す ID (ゾーン ID), 各乗車客の乗車時間, 下車時間, 走行距離, 支払い料金, チップ, 総支払い料金などがある. ここでは乗車ゾーン ID ごとに総支払い料金の平均値を求め, 衛星画像上にその値を色による強度としてマッピングする. 本実験はメモリに収まりきらないデータの並列・分散処理を見据えた実験である.

以下のスクリプトはフィールド名が PULocationID と total_amount のデータ（それぞれは乗車したゾーンの ID と総支払い料金）を抽出し、ゾーンごとの支払い料金の平均値を求めるものである。MapReduce を逐次処理で行う場合と並列・分散処理で行う場合のスクリプトを示す（program_m1.m と program_m2.m）。MapReduce の場合、メインのスクリプトに加え、map 関数と reduce 関数が必要になる。

```
% program_m1.m

clear
addpath('/cis/public/datasci/jikken/L8data');

inMatlab=mapreducer(0);

tdir='/cis/public/datasci/class/taxi_yellow/2017/';
path1={['tdir 'yellow_tripdata_2017-01.csv'],...
        ['tdir 'yellow_tripdata_2017-02.csv']};

tic
ds=datastore(path1);
ds.SelectedVariableNames={'PULocationID','total_amount'};

ticBytes(gcp)
output=mapreduce(ds,@statmapper,@statreducer,inMatlab);
tocBytes(gcp)

r=readall(output); % Read output data
id=1:265; % Define number of zone
a=[id;r.Value{:}.fare]; % Create array of ID and fare
tmp=r.Value{:}.num<100; % Identify index number for (data size) < 100
fare_ave=a(2,:);
fare_ave(tmp)=nan; % Remove data for (data size) < 100
delete result*
elpt=toc
```



```
% program_m2.m

clear
addpath('/cis/public/datasci/jikken/L8data');

p=parpool(16);
inPool=mapreducer(p);

tdir='/cis/public/datasci/class/taxi_yellow/2017/';
path1={['tdir 'yellow_tripdata_2017-01.csv'],...
        ['tdir 'yellow_tripdata_2017-02.csv']];

tic
ds=datastore(path1);
ds.SelectedVariableNames={'PULocationID','total_amount'};

ticBytes(gcp)
output=mapreduce(ds,@statmapper,@statreducer,inPool);
tocBytes(gcp)

r=readall(output);
id=1:265;
a=[id;r.Value{:}.fare];
tmp=r.Value{:}.num<100;
fare_ave=a(2,:);
fare_ave(tmp)=nan;
delete result*
elpt=toc
```

以下は map 関数である.

```
function statmapper(data,~,intermKVStore)

fare=data.total_amount;
puid=data.PULocationID;

tmpid=fare>300|fare<=0; %Identify index number for anomaly
fare(tmpid)=[]; % Remove data for anomaly
puid(tmpid)=[]; % Remove data for anomaly

for z=1:265
    tmp=puid==z;
    tmp2=fare(tmp);
    fare_sum(z)=sum(tmp2); % Calculate sum of fare for each zone
    fare_num(z)=size(tmp2,1); % Calculate number of sample for each zone
end
% Save results of map function that is subsequently read in reduce function
add(intermedKVStore,'key',{[fare_sum' fare_num]});
```

以下は reduce 関数である.

```

function statreducer(~,intermValIter,outKVStore)

tmpfare=[];tmpnum=[];
c=0;
while hasnext(intermValIter) % Check if results from map function still exist
    tmp=getnext(intermValIter); % Get results from map function
    tmp2=tmp{:}; % Cell array to double array
    c=c+1;
    tmpfare(c,:)=tmp2(:,1);
    tmpnum(c,:)=tmp2(:,2);
end

stat.fare=sum(tmpfare,1)./sum(tmpnum,1); % Calculate averaged fare of each zone
stat.num=sum(tmpnum,1); % Calculate total number of each zone

add(outKVStore,'count',stat); % Save output
end

```

各スクリプトで得られた結果は以下のスクリプトで可視化することができる。ただし、`paint_shape` は PULocationID ごとにデータを色を付けて衛星画像上に表示させるスクリプトであり、値を 0 から 1 に規格化して色付けしている（最小値が 0、最大値が 1 となるようにスケーリングをしている）。表示のための関連ファイルは最初に定義したパスで指定されるディレクトリ（`/cis/public/datasci/jikken/L8data`）に置かれているが、実験時には特に気にする必要はない。そのため、以下の `mapimage.m` のみ作成すればよい。例えば、配列 `fare_ave` を可視化したい場合、コマンドラインで「`mapimage(fare_ave,id)`」とタイプする。

```

% mapimage.m

function mapimage(param,id)

lw=1;
load colorv
load mat4im

input=[id' param'/(max(param))];

tmp2=isnan(input(:,2));
input(tmp2,:)=[];

figure(2);
mapshow(rlon_mat,rlat_mat,rJ);hold on
S=shaperead('Export_Output');
for z=1:size(S,1)
    z
    mapshow(S(z),'facecolor','none','edgecolor','k',...
        'linewidth',lw);hold on;
end
hold on;
paint_shape(input,S,lw,c);
set(gca,'fontsize',10);colormap(jet);colorbar;
xlabel('Longitude');ylabel('Latitude');
hold off;

```

タクシーデータの MapReduce を用いた並列・分散処理に関する以下の課題を行え。ただし、時間の計測には `tic` と `toc` を利用すること。さらに、逐次処理や各ワーカー数での並列・分散処理における時間の計測時は、二回以上行い最も時間の短いものを記録すること。

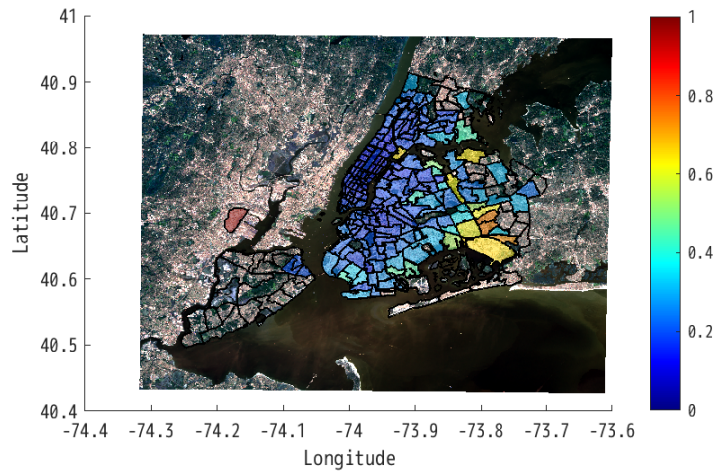


図 8: 課題 1 の処理結果に対するゾーン（区画）ごとの総支払い料金平均値の可視化例.

作業 1 program_m1.m, map 関数, reduce 関数, mapimage.m を作成して MapReduce の逐次処理を行い, 処理時間を計測せよ.

作業 2 program_m2.m を作成して MapReduce の並列処理を, 2, 4, 6, 8, 12, 16 並列で行い, 処理時間を計測せよ. 以下, 作業手順の例を示す.

- 並列数を指定の値に設定
- tic 以降を選択して実行
- もう一度 tic 以降を選択して実行, 時間の計測 (JIT コンパイル機能を活かして高速化するため)
- delete(gcp) で並列プールを削除

作業 3 program_m2.m 実行後, コマンドラインで mapimage(fare_ave,id) とタイプして図 8 のように処理結果 (fare_ave) が衛星画像上に表示されることを確かめよ (結果の見方は mapimage.m の部分の説明を参照.) ただし, 図示した結果は報告しなくてよい (レポートにも記載しなくてよい).

【課題 1】

以上の作業により得られた結果をもとに, 以下の課題を行え.

課題 1-1 作業 1,2 の結果を利用して, コア数毎 ($N \geq 2$) に, 計測時間, 速度向上度 (s_n), Karp-Flatt metric (K_f) を計算し, 結果を折れ線グラフ (横軸は並列数) でプロットせよ.

$$K_f = \frac{\frac{N}{s_n} - 1}{N - 1} \quad (24)$$

$$s_n = \frac{t_s}{t_n} \quad (25)$$

補足 上式の t_s は逐次処理時間, t_n は N 並列による処理時間であり, 配布スクリプトで計測される時間と考えてよい.

課題 1-2 MapReduce のアルゴリズムの利点・欠点について調査せよ.

【課題 2】

次に, データの分析に関する以下の課題を行え (可視化して得られた図はレポートに含めること). 16 並列による MapReduce 処理を利用すると良い.

課題 2-1 program_m2.m をコピーして program_m3.m を作成し, 読み込み対象のデータを 2017 年 1 月から 12 月までとなるよう修正せよ. その後, 処理結果 (fare_ave) を mapimage.m により可視化して得られる図を示せ. また, その図と地域の土地利用に関する特徴を調べて見比べ, 今回の結果が得られた理由を考察せよ. ただし, 土地利用に関する特徴は, GoogleMap 等のウェブ上で閲覧可能な地図アプリケーションなどを利用して良い.

program_m3.m ではパスのみを書き換える.

```
path1={[tdir 'yellow_tripdata_2017-01.csv'],...
        [tdir 'yellow_tripdata_2017-02.csv'],...
        [tdir 'yellow_tripdata_2017-03.csv'],...
        [tdir 'yellow_tripdata_2017-04.csv'],...
        [tdir 'yellow_tripdata_2017-05.csv'],...
        [tdir 'yellow_tripdata_2017-06.csv'],...
        [tdir 'yellow_tripdata_2017-07.csv'],...
        [tdir 'yellow_tripdata_2017-08.csv'],...
        [tdir 'yellow_tripdata_2017-09.csv'],...
        [tdir 'yellow_tripdata_2017-10.csv'],...
        [tdir 'yellow_tripdata_2017-11.csv'],...
        [tdir 'yellow_tripdata_2017-12.csv']};
```

課題 2-2 2017 年 1 月から 12 月までのデータを対象に, trip_distance (走行距離) のデータを用いて単位距離当たりの料金 (料金の総和を距離の総和で除したもの) を PULocationID ごとに求め, mapimage.m により可視化せよ. また, 可視化した結果と地域の土地利用に関する特徴を調べ, 今回の結果が得られた理由を考察せよ. ただし, 支払い料金が 0 ドル以下または 300 ドルより大きい, または, 走行距離が 0.5km 以下または 100km より大きいものは除外すること. スクリプトの補足は以下を参照のこと. メイン関数の修正箇所は, 課題 2-1 の ds.SelectedVariableNames の項目に trip_distance を追加. また, 下から 6 行目の a の定義部分で, fareperdist を参照. mapreduce の引数である map 関数と reduce 関数の名称を変更. map 関数と reduce 関数の例は以下の通り.

```

function statmapper2(data,~,intermKVStore)

fare=data.total_amount;
puid=data.PULocationID;
dist=data.trip_distance;

tmpid=fare>300|fare<=0|dist<0.5|dist>100;
fare(tmpid)=[];
puid(tmpid)=[];
dist(tmpid)=[];

for z=1:265
    tmp=puid==z;
    tmp2=fare(tmp);
    tmp3=dist(tmp);
    fare_sum(z)=sum(tmp2);
    fare_num(z)=size(tmp2,1);
    dist_sum(z)=sum(tmp3);
end

add(intermapKVStore,'key',{[fare_sum' fare_num' dist_sum']});

```

```

function statreducer2(~,intermValIter,outKVStore)

tmpfare=[];tmpnum=[];tmpdist=[];
c=0;
while hasnext(intermapValIter)
    tmp=getnext(intermapValIter);
    tmp2=tmp{:};
    c=c+1;
    tmpfare(c,:)=tmp2(:,1);
    tmpnum(c,:)=tmp2(:,2);
    tmpdist(c,:)=tmp2(:,3);
end

stat.fareperdist=sum(tmpfare,1)./sum(tmpdist,1);
stat.num=sum(tmpnum,1);

add(outKVStore,'count',stat);
end

```

参考文献

- [1] 工藤卓哉, 保科学世, "データ・アナリティクス実践講座," 翔泳社, 2016.
- [2] 酒巻隆治, 里洋平, "ビジネス活用事例で学ぶデータサイエンス入門," SB クリエイティブ株式会社, 2014.
- [3] 総務省 (編), "平成 24 年版情報通信白書," 総務省, 2012.
- [4] Manyika, J, et al., "Big data : The next frontier for innovation, competition, and productivity," McKinsey Global Institute, 2011.
- [5] Mathworks 日本, "ビッグデータ - MATLAB and Simlink," available online, <https://jp.mathworks.com/discovery/big-data-matlab.html> (accessed on 2017/09/18).
- [6] 中井悦司, "機械学習理論入門," 技術評論社, 2016.
- [7] 吉田琢也, "すべてがわかるビッグデータ大全," 日経 BP, 2016.
- [8] Antonio J. Plaza, "High Performance Computing in Remote Sensing," CHAPMAN and HALL/CRC, 2008.
- [9] 海老原熊雄, 大森健児訳, "並列プログラミングの基礎," 丸善, 1990.
- [10] Gregory R. Andrews, "Concurrent Programming," The Benjamin/Cummings Publishing Company, Inc., 1991.
- [11] H. Stephen Morse, "Practical Parallel Computing," Academic Press, Inc., 1994.
- [12] Kevin Dowd, Charles R. Severance, "High Performance Computing," O'REILLY, 1998.
- [13] Clay Breshears, 千住治郎訳, "並列コンピューティング技法-実践マルチコア/ マルチスレッドプログラミング," オーム社, 2010.
- [14] 安田絹子, 小林林広, 飯塚博道, 阿部貴之, 青柳信吾, "マルチコア CPU のための並列プログラミング," 秀和システム, 2008.
- [15] Anon., "Parallel Computing Toolbox 5 User's Guide," The MathWorks, Inc., available online, <http://www.mathworks.com/help/toolbox/distcomp/bqqaxq0.html>, 2010.
- [16] Jermy Kepner, "Parallel MATLAB for Multicore and Multinode Computers," Society for Industrial and Applied Mathematics Philadelphia, 2009.
- [17] Zbigniew J. Czech, "Introduction to Parallel Computing," Cambridge University Press, 2016.
- [18] Xing Cai, "The Karp-Flatt metric; revised," available online, https://www.uio.no/studier/emner/matnat/ifi/INF3380/v12/undervisningsmateriale/correct_Karp_Flatt.pdf (accessed on 2021/7/15).
- [19] Dean, J. and Ghemawat, S., "MapReduce: Simplified Data Processing on Large Clusters," Research, Google, 2004.
- [20] Mathworks 日本, "MapReduce 入門, - MATLAB spmd -, " available online, https://jp.mathworks.com/help/matlab/import_export/getting-started-with-mapreduce.html (accessed on 2018/02/24).
- [21] Mathworks 日本, "Tall 配列," available online, <https://jp.mathworks.com/help/matlab/tall-arrays.html> (accessed on 2018/02/24).
- [22] NOAA, "The Integrated Surface Database (ISD)," available online at, <https://www.ncdc.noaa.gov/isd/data-access> (accessed on 2018/02/24).
- [23] City of New York, "TLC Trip Data," available online, <https://www1.nyc.gov/site/tlc/about/tlc-trip-record-data.page> (accessed on 2021/07/26).
- [24] City of New York, "Data Dictionary - Yellow Taxi Trip Records," available online, https://www1.nyc.gov/assets/tlc/downloads/pdf/data_dictionary_trip_records_yellow.pdf (accessed on 2021/07/26).